

AI 工具使用声明

本队在竞赛过程中使用 AI 工具辅助资料检索、难点思路讨论、程序编写。AI 生成内容仅作参考，都经过回查题目与附件、检查模型与代码、核验计算结果。

参考文献

- [1] National Institute of Standards and Technology. NIST Digital Library of Mathematical Functions[EB/OL]. Sections 10.6, 10.22[2026-09-12]. <https://dlmf.nist.gov/>.
- [2] Shampine L F, Reichelt M W. The MATLAB ODE Suite[J]. *SIAM Journal on Scientific Computing*, 1997, 18(1): 1–22. DOI: 10.1137/S1064827594276424.
- [3] Virtanen P, Gommers R, Oliphant T E, et al. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python[J]. *Nature Methods*, 2020, 17(3): 261–272. DOI: 10.1038/s41592-019-0686-2.
- [4] Teleken J T, Amorim S M, Rodrigues S S S, et al. Heat and Mass Transfer in Shrimp Hot-Air Drying: Experimental Evaluation and Numerical Simulation[J]. *Foods*, 2025, 14(3): 428. DOI: 10.3390/foods14030428.
- [5] Brasiello A, Venditti C, Adrover A. Non-Isothermal Moving-Boundary Model for Food Drying[J]. *Chemical Engineering Transactions*, 2021, 87: 193–198. DOI: 10.3303/CET2187033.
- [6] Adrover A, Venditti C, Brasiello A. A Non-Isothermal Moving-Boundary Model for Continuous and Intermittent Drying of Pears[J]. *Foods*, 2020, 9(11): 1577. DOI: 10.3390/foods9111577.

附录 A 文件清单

支撑材料中的程序、数据、图件与说明文件分别列于下表。除已注明的公共目录外，路径均相对于支撑材料根目录。

表 1: 程序文件（公共目录：code/）

文件名	功能说明
common.py	原始观测读取、环境与半径插值
export_results.py	四份结果工作簿导出
extract_figure_data.py	绘图数值样本提取
figure_coordinates.py	实测收缩与材料坐标图
figure_core.py	问题一径向剖面图
figure_fields.py	问题二时空分布图
figure_restructured.py	空间离散、扩散系数分解与达标进程图
figure_sections.py	问题四含水率截面图
q1_model.py	问题一径向有限体积计算
q23_model.py	问题二、三的局部变物性耦合计算
q4_model.py	问题四收缩域耦合计算
redraw_figures.py	数据图绘制入口
requirements.txt	公开软件依赖及版本
requirements_figures.txt	绘图软件依赖及版本
run_models.py	问题选择、参数设置与限时运行入口

表 2: 输入数据、绘图数据与计算结果

文件名	内容
data/templates/result1.xlsx	题给 result1 输出模板
data/templates/result2.xlsx	题给 result2 输出模板
data/templates/result3.xlsx	题给 result3 输出模板
data/templates/result4.xlsx	题给 result4 输出模板
data/附件 1.xlsx	题给环境温度与水分浓度观测
data/附件 2.xlsx	题给半径收缩观测
figure_data/data_manifest.json	绘图数值样本及其来源、单位说明
figure_data/q1_profiles.npz	绘图数值样本及其来源、单位说明
figure_data/q2_fields.npz	绘图数值样本及其来源、单位说明
figure_data/q4_sections.npz	绘图数值样本及其来源、单位说明
figure_data/radius_observations.json	绘图数值样本及其来源、单位说明
figure_data/restructured_figure_data.json	绘图数值样本及其来源、单位说明
results/result1.xlsx	问题一：1 秒间隔、前 30 分钟
results/result2.xlsx	问题二：1 秒间隔、前三小时

表 2 输入数据、绘图数据与计算结果（续）

文件名	内容
results/result3.xlsx	问题三：1 分钟间隔与完整干燥终点
results/result4.xlsx	问题四：收缩过程、固定位置与实际表面

图件均位于 **figures/**，完整文件名由表中基名加对应扩展名构成。

表 3: 论文图件文件

文件基名	扩展名	对应图号
fig1_1_fig01_annular_control_volumes	pdf / svg / png	图 1-1
fig1_2_fig01_spacetime_grid	pdf / svg / png	图 1-2
fig1_3_fig03_q1_profiles	pdf / svg	图 1-3
fig1_3_fig03_q1_profiles_d98d193909de	png	图 1-3
fig2_1_fig04_q2_fields_sample	pdf / svg / png	图 2-1
fig2_2_fig09_q2_diffusivity_decomposition	pdf / svg / png	图 2-2
fig3_1_fig05_q3_completion_progress	pdf / svg / png	图 3-1
fig4_1_fig06_q4_coordinates	pdf / svg / png	图 4-1
fig4_2_fig07_q4_sections_sample	pdf / svg / png	图 4-2

表 4: 参考文献与说明文件

文件名	用途
AI 工具使用详情.pdf	AI 使用范围与人工修订说明
figures/figure_manifest.json	图号、题注、文件对应关系
figures/README.md	图件编辑与重绘说明
README.md	环境配置、运行方法与文件说明
references/references.bib	参考文献题录与检索标识
references/references.txt	参考文献题录与检索标识

附录 B 完整程序

以下按逐问模型、公共模块和绘图程序列出源代码。题给附录 3、4 分别指原题提供的物性关系。版面中的长行自动续排，行号仍按原始源程序计数。

B.1 问题一：预热阶段的径向传热传质

给出节点控制体、通量、解析 Jacobian 及分段隐式积分的完整实现。

程序 1 code/q1_model.py

```
1  """Q1: fixed-radius nodal finite volumes; independent temperature and moisture."""
2  from __future__ import annotations
3  import math
4  import time
5  from pathlib import Path
6  import numpy as np
7  from scipy.integrate import solve_ivp
8  from scipy.sparse import coo_matrix
9  from common import Boundary, RadiusInput, save_result
10
11  R=0.02
12  RHO,CP,K,H,HM=820.0,2600.0,0.36,25.0,8e-7
13  SNAPSHOT_TIMES=np.array([0,1,60,1800],dtype=int)
14
15  class FieldSystem:
16      """N+1 physical nodes and one diagnostic integrated boundary-flux state."""
17
18      def __init__(self, n, field, boundary, constant_diffusivity=False):
19          if not isinstance(n, (int, np.integer)) or n < 20 or n % 20:
20              raise ValueError("N must be a positive multiple of 20")
21          if field not in ("temperature", "moisture"):
22              raise ValueError("Unknown field")
23          self.n, self.field, self.boundary = n, field, boundary
24          self.constant_diffusivity = constant_diffusivity
25          self.r = np.linspace(0, R, n + 1)
26          self.dr = R / n
27          faces = (self.r[:-1] + self.r[1:]) / 2
28          edges = np.r_[0.0, faces, R]
29          self.v = np.diff(edges**2) / 2
30          self.q = faces / self.dr
31          self.capacity = RHO * CP if field == "temperature" else 1.0
32          self.transfer = H if field == "temperature" else HM
33          self.initial = 28.0 if field == "temperature" else 2.55
34
35      def coefficients(self, u):
36          if not np.isfinite(u).all():
37              raise ValueError("Nonfinite field state")
38          if self.field == "temperature":
39              return np.full_like(u, K), np.zeros_like(u)
```

程序 1 code/q1_model.py (续)

```

40     if np.any(u <= 0):
41         raise ValueError("Nonpositive moisture state; clipping is forbidden")
42     if self.constant_diffusivity:
43         return np.full_like(u, 7e-9 * np.exp(-0.89 / 2.55)), np.zeros_like(u)
44     d = 7e-9 * np.exp(-0.89 / u)
45     return d, 0.89 * d / u**2
46
47     def rhs(self, t, y):
48         u = y[:-1]
49         d, _ = self.coefficients(u)
50         g = self.boundary.evaluate(t)[0 if self.field == "temperature" else 1]
51         flux = np.empty(self.n + 2)
52         flux[0] = 0.0
53         flux[1:-1] = self.q * (d[:-1] + d[1:]) / 2 * np.diff(u)
54         flux[-1] = R * self.transfer * (g - u[-1])
55         return np.r_[np.diff(flux) / (self.capacity * self.v), flux[-1]]
56
57     def jac(self, t, y):
58         u = y[:-1]
59         d, dp = self.coefficients(u)
60         delta = np.diff(u)
61         dbar = (d[:-1] + d[1:]) / 2
62         a = self.q * (0.5 * dp[:-1] * delta - dbar)
63         b = self.q * (0.5 * dp[1:] * delta + dbar)
64         weights = self.capacity * self.v
65         ix = np.arange(self.n)
66         rows = np.r_[ix, ix, ix + 1, ix + 1, self.n, self.n + 1]
67         cols = np.r_[ix, ix + 1, ix, ix + 1, self.n, self.n]
68         vals = np.r_[a / weights[:-1], b / weights[:-1],
69                     -a / weights[1:], -b / weights[1:],
70                     -R * self.transfer / weights[-1], -R * self.transfer]
71         return coo_matrix((vals, (rows, cols)), shape=(self.n + 2, self.n + 2)).
72         tosc()
73
74     def solve_field(n, field, boundary, rtol=2e-9, atol_scale=1.0,
75                   constant_diffusivity=False):
76         started = time.perf_counter()
77         system = FieldSystem(n, field, boundary, constant_diffusivity)
78         y = np.r_[np.full(n + 1, system.initial), 0.0]
79         index = np.arange(21) * (n // 20)
80         values = np.empty((1801, 21))
81         mean = np.empty(1801)
82         integral = np.empty(1801)
83         snapshots = np.empty((4, n + 1))
84         integral_snapshots = np.empty(4)
85         values[0], mean[0], integral[0] = system.initial, system.initial, 0.0
86         snapshots[0], integral_snapshots[0] = y[:-1], y[-1]
87         base_atol = (2e-9 if field == "temperature" else 2e-11) * atol_scale

```

程序1 code/q1_model.py (续)

```

88     atol = np.r_[np.full(n + 1, base_atol), base_atol * system.capacity * R**2 /
89     2]
89     nfev = njev = nlu = 0
90     for a in range(0, 1800, 60):
91         sample_times = np.arange(a + 1, a + 61)
92         sol = solve_ivp(system.rhs, (a, a + 60), y, method="BDF", jac=system.jac,
93             rtol=rtol, atol=atol, max_step=5.0, t_eval=sample_times)
94         if not sol.success or sol.y.shape != (n + 2, 60):
95             raise RuntimeError(f"{field} N={n}, interval {a}: {sol.message}")
96         if not np.isfinite(sol.y).all() or (field == "moisture" and np.any(sol.y
97        [:-1] <= 0)):
98             raise RuntimeError("Invalid solution; no successful output will be
99             written")
100         values[sample_times] = sol.y[index].T
101         mean[sample_times] = 2 / R**2 * (system.v @ sol.y[:-1])
102         integral[sample_times] = sol.y[-1]
103         for j, t in enumerate(SNAPSHOT_TIMES[1:], 1):
104             if a < t <= a + 60:
105                 snapshots[j] = sol.y[:-1, t - a - 1]
106                 integral_snapshots[j] = sol.y[-1, t - a - 1]
107             y = sol.y[:, -1]
108             nfev += sol.nfev
109             njev += sol.njev
110             nlu += sol.nlu
111         return dict(values=values, mean=mean, integral=integral, snapshots=snapshots,
112             integral_snapshots=integral_snapshots, n=n, nfev=nfev, njev=njev,
113             nlu=nlu, elapsed_s=time.perf_counter()-started)
114
115 def run(n, output, data, *, tight=False):
116     boundary=Boundary(Path(data)/"附件1.xlsx",q1=True)
117     started=time.perf_counter()
118     fields={name:solve_field(n,name,boundary,rtol=2e-10 if tight else 2e-9,
119         atol_scale=0.1 if tight else 1.0)
120         for name in ("temperature","moisture")}
121     payload=dict(time_s=np.arange(1801),radius_m=np.linspace(0,R,21),
122         mesh_radius_m=np.linspace(0,R,n+1),snapshot_time_s=SNAPSHOT_TIMES
123         ,
124         temperature_C=fields["temperature"]["values"],
125         moisture_kg_kg=fields["moisture"]["values"])
126     for name,result in fields.items():
127         for key in ("mean","integral","snapshots","integral_snapshots"):
128             payload[name+"_"+key]=result[key]
129     metadata=dict(grid_n=n,tight=tight,elapsed_s=time.perf_counter()-started,
130         max_step_s=5.0,rtol=2e-10 if tight else 2e-9,
131         atol_temperature=2e-10 if tight else 2e-9,
132         atol_moisture=2e-12 if tight else 2e-11)
133     return save_result(output,"q1",payload,metadata)

```

B.2 问题二、三：局部变物性耦合计算

给出题给附录 3 物性、热质耦合方程及全域含水率达标判据。

程序 2 code/q23_model.py

```
1  """Q2/Q3: local coupled properties and a full-domain drying criterion."""
2  from __future__ import annotations
3  import math
4  import time
5  from pathlib import Path
6  import numpy as np
7  from scipy.integrate import solve_ivp
8  from scipy.sparse import coo_matrix
9  from common import Boundary, RadiusInput, save_result
10
11  R,H,HM=0.02,25.0,8e-7
12  INITIAL_T,INITIAL_C,THRESHOLD=28.0,2.55,0.15
13  OBSERVED_END=14400
14  SUMMARY_TIMES=(1800,3600,5400,7200,9000,10800)
15  MAX_EXCEL_DATA_ROWS=1048575
16  SAMPLE_BLOCK=64
17
18  class System:
19      """Interleaved (theta_0,C_0,...,theta_N,C_N), with no diagnostic states."""
20
21      def __init__(self, n, boundary):
22          if isinstance(n, (bool, np.bool_)) or not isinstance(n, (int, np.integer))
23          or n < 1:
24              raise ValueError("N must be a positive integer")
25          self.n, self.boundary = int(n), boundary
26          self.r = np.linspace(0.0, R, self.n + 1)
27          self.dr = R / self.n
28          faces = (self.r[:-1] + self.r[1:]) / 2
29          self.v = np.diff(np.r_[0.0, faces, R] ** 2) / 2
30          self.q = faces / self.dr
31          self.h, self.hm = H, HM
32          self.size = 2 * (self.n + 1)
33
34      def split(self, y):
35          y = np.asarray(y)
36          if y.shape != (self.size,) or not np.isfinite(y).all():
37              raise ValueError("Invalid joint-state shape or nonfinite value")
38          theta, c = y[0::2], y[1::2]
39          # Real-part checks permit an infinitesimal complex-step oracle without
40          # modifying the analytic formulas. Production integration uses float64.
41          if np.any(c.real <= 0) or np.any(theta.real <= -273.15):
42              raise ValueError("Nonpositive moisture or absolute temperature; no
43              clipping")
44          return theta, c
45
46      @staticmethod
```

程序 2 code/q23_model.py (续)

```

45 def coefficients(theta, c):
46     rho = 650.0 + 128.0 * c
47     cp = 1450.0 + 2736.0 * c / (1.0 + c)
48     k = 0.21 + 0.38 * c / (1.0 + c)
49     d = 2.4e-3 * np.exp(-0.45 / c - 3850.0 / (theta + 273.15))
50     return dict(rho=rho, cp=cp, s=rho * cp, k=k, D=d,
51                 s_C=128.0 * cp + rho * 2736.0 / (1.0 + c)**2,
52                 k_C=0.38 / (1.0 + c)**2,
53                 D_C=0.45 * d / c**2,
54                 D_T=3850.0 * d / (theta + 273.15)**2)
55
56 def powers(self, t, theta, c, p):
57     dtype = np.result_type(theta, c, float)
58     ft = np.empty(self.n + 2, dtype=dtype)
59     fc = np.empty(self.n + 2, dtype=dtype)
60     ft[0] = fc[0] = 0
61     ft[1:-1] = self.q * (p["k"][:-1] + p["k"][1:]) / 2 * np.diff(theta)
62     fc[1:-1] = self.q * (p["D"][:-1] + p["D"][1:]) / 2 * np.diff(c)
63     t_air, c_eq = self.boundary.evaluate(t)
64     ft[-1] = R * self.h * (t_air - theta[-1])
65     fc[-1] = R * self.hm * (c_eq - c[-1])
66     return np.diff(ft), np.diff(fc)
67
68 def rhs(self, t, y):
69     theta, c = self.split(y)
70     p = self.coefficients(theta, c)
71     qt, qc = self.powers(t, theta, c, p)
72     out = np.empty(self.size, dtype=np.result_type(y, float))
73     out[0::2] = qt / (self.v * p["s"])
74     out[1::2] = qc / self.v
75     return out
76
77 def jac(self, t, y):
78     theta, c = self.split(y)
79     p = self.coefficients(theta, c)
80     dt, dc = np.diff(theta), np.diff(c)
81     kb = (p["k"][:-1] + p["k"][1:]) / 2
82     db = (p["D"][:-1] + p["D"][1:]) / 2
83     # Each column is the derivative with respect to theta_i, C_i, theta_j, C_j.
84     jt = self.q[:, None] * np.column_stack(
85         (-kb, p["k_C"][:-1] * dt / 2, kb, p["k_C"][1:] * dt / 2))
86     jc = self.q[:, None] * np.column_stack(
87         (p["D_T"][:-1] * dc / 2, p["D_C"][:-1] * dc / 2 - db,
88          p["D_T"][1:] * dc / 2, p["D_C"][1:] * dc / 2 + db))
89     i = np.arange(self.n)
90     cols_face = 2 * i[:, None] + np.arange(4)[None, :]
91     rows, cols, vals = [], [], []
92     for offset, face_jac, weights in ((0, jt, self.v * p["s"]), (1, jc, self.v
93 )):
94         for shift, sign in ((0, 1), (1, -1)):

```

程序2 code/q23_model.py (续)

```

94         nodes = i + shift
95         rows.append(np.repeat(2 * nodes + offset, 4))
96         cols.append(cols_face.ravel())
97         vals.append((sign * face_jac / weights[nodes, None]).ravel())
98     qt, _ = self.powers(t, theta, c, p)
99     theta_dot = qt / (self.v * p["s"])
100    nodes = np.arange(self.n + 1)
101    rows.extend([2 * nodes, np.array([2 * self.n, 2 * self.n + 1])])
102    cols.extend([2 * nodes + 1, np.array([2 * self.n, 2 * self.n + 1])])
103    vals.extend([-p["s_C"] / p["s"] * theta_dot,
104                np.array([-R * self.h / (self.v[-1] * p["s"][-1]),
105                          -R * self.hm / self.v[-1]])])
106    return coo_matrix((np.concatenate(vals), (np.concatenate(rows), np.
107        concatenate(cols))),
108                      shape=(self.size, self.size)).tocsc()
109
110    class Recorder:
111        """Only short dense blocks, output columns, and selected full states live here
112        ."""
113
114        def __init__(self, system, boundary):
115            self.system = system
116            self.output_nodes = np.arange(21) * (system.n // 20)
117            self.temperature, self.moisture = [], []
118            self.maxima, self.argmax = [], []
119            self.t_mean, self.c_mean = [], []
120            self.last_time = -1
121            self.last_state = None
122            self.snapshots, self.roles = {}, {}
123            self.n_end = None
124            self.extrema = dict(temperature_min_C=float("inf"), temperature_max_C=-
125                float("inf"),
126                               moisture_min_kg_kg=float("inf"), moisture_max_kg_kg=-
127                float("inf"))
128            self.bounds = dict(temperature_min_C=min(INITIAL_T, float(boundary.
129                temperature.min()), boundary.tail[0]),
130                               temperature_max_C=max(INITIAL_T, float(boundary.
131                temperature.max()), boundary.tail[0]),
132                               moisture_min_kg_kg=min(INITIAL_C, float(boundary.
133                moisture.min()), boundary.tail[1]),
134                               moisture_max_kg_kg=max(INITIAL_C, float(boundary.
135                moisture.max()), boundary.tail[1]))
136            self.allowances = dict(temperature_C=1e-7, moisture_kg_kg=1e-8)
137            self.fixed_roles = {0: "initial", OBSERVED_END: "observed_end"}
138            self.fixed_roles.update({t: f"summary_{t}s" for t in SUMMARY_TIMES})
139
140        def inspect(self, states):
141            a = np.asarray(states)

```

程序 2 code/q23_model.py (续)

```

135         if not np.isfinite(a).all() or np.any(a[1::2] <= 0) or np.any(a[0::2] <=
136         -273.15):
137             raise RuntimeError("Accepted solution contains nonfinite or
138             nonphysical values")
139             for field, values, allowance in (("temperature", a[0::2], 1e-7),
140             ("moisture", a[1::2], 1e-8)):
141                 suffix = "C" if field == "temperature" else "kg_kg"
142                 low, high = f"{field}_min_{suffix}", f"{field}_max_{suffix}"
143                 self.extrema[low] = min(self.extrema[low], float(values.min()))
144                 self.extrema[high] = max(self.extrema[high], float(values.max()))
145                 if values.min() < self.bounds[low] - allowance or values.max() > self.
146                 bounds[high] + allowance:
147                     raise RuntimeError(f"Accepted {field} exceeds the stated invariant
148                     -range allowance")
149
150     def snapshot(self, t, y, role):
151         self.snapshots[float(t)] = np.asarray(y, dtype=float).copy()
152         self.roles[role] = float(t)
153
154     def append(self, times, states):
155         times = np.asarray(times, dtype=np.int64)
156         if times.size == 0:
157             return
158         if not np.array_equal(times, np.arange(self.last_time + 1, self.last_time
159         + 1 + len(times))):
160             raise RuntimeError("Missing, repeated, or unordered integer output
161             sample")
162         self.inspect(states)
163         theta, c = states[0::2], states[1::2]
164         maxima = c.max(axis=0)
165         if self.n_end is None:
166             below = np.flatnonzero((times > 0) & (maxima < THRESHOLD))
167             if below.size:
168                 j = int(below[0])
169                 self.n_end = int(times[j])
170                 preceding = states[:, j - 1] if j else self.last_state
171                 if preceding is None or np.max(preceding[1::2]) < THRESHOLD:
172                     raise RuntimeError("Cannot establish the first strict integer
173                     threshold sample")
174                 self.snapshot(self.n_end - 1, preceding, "before_n_end")
175                 self.snapshot(self.n_end, states[:, j], "n_end")
176         for j, t in enumerate(times):
177             if int(t) in self.fixed_roles:
178                 self.snapshot(t, states[:, j], self.fixed_roles[int(t)])
179         self.temperature.append(theta[self.output_nodes].T.copy())
180         self.moisture.append(c[self.output_nodes].T.copy())
181         self.maxima.append(maxima.copy())
182         self.argmax.append(c.argmax(axis=0).astype(np.int64))
183         self.t_mean.append(2 / R**2 * (self.system.v @ theta))
184         self.c_mean.append(2 / R**2 * (self.system.v @ c))

```

程序 2 code/q23_model.py (续)

```

178         self.last_time, self.last_state = int(times[-1]), states[:, -1].copy()
179
180     def finish(self):
181         self.snapshot(self.last_time, self.last_state, "common_end")
182         snapshot_times = sorted(self.snapshots)
183         snapshots = np.asarray([self.snapshots[t] for t in snapshot_times])
184         nodes = np.concatenate(self.argmax)
185         payload = dict(time_s=np.arange(self.last_time + 1, dtype=np.int64),
186                        radius_m=self.system.r[self.output_nodes].copy(),
187                        mesh_radius_m=self.system.r.copy(),
188                        temperature_C=np.concatenate(self.temperature),
189                        moisture_kg_kg=np.concatenate(self.moisture),
190                        max_moisture_kg_kg=np.concatenate(self.maxima),
191                        max_moisture_node=nodes,
192                        max_moisture_radius_m=self.system.r[nodes],
193                        temperature_mean=np.concatenate(self.t_mean),
194                        moisture_mean=np.concatenate(self.c_mean),
195                        snapshot_time_s=np.asarray(snapshot_times),
196                        temperature_snapshots=snapshots[:, 0::2],
197                        moisture_snapshots=snapshots[:, 1::2])
198         return payload, {role: snapshot_times.index(t) for role, t in self.roles.
199                          items()}
200
201     def solve(n, boundary, *, tight=False, common_end_s=None, max_time_s=
202             MAX_EXCEL_DATA_ROWS):
203         """Return (NPZ payload, run diagnostics); caller owns persistence and
204         acceptance."""
205         if isinstance(n, (bool, np.bool_)) or not isinstance(n, (int, np.integer)) or
206         n < 20 or n % 20:
207             raise ValueError("Production N must be a positive multiple of 20")
208         for name, value in (("common_end_s", common_end_s), ("max_time_s", max_time_s)
209                             ):
210             if value is not None and (isinstance(value, bool) or not isinstance(value,
211                                         (int, np.integer))) or value < 1:
212                 raise ValueError(f"{name} must be a positive integer")
213             if max_time_s < OBSERVED_END or (common_end_s is not None and common_end_s >
214                 max_time_s):
215                 raise ValueError("Work limit cannot cover the requested output interval")
216             started = time.perf_counter()
217             system = System(n, boundary)
218             recorder = Recorder(system, boundary)
219             y = np.tile([INITIAL_T, INITIAL_C], n + 1)
220             recorder.append([0], y[:, None])
221             factor = 0.1 if tight else 1.0
222             rtol, atol_t, atol_c = 2e-9 * factor, 2e-9 * factor, 2e-11 * factor
223             atol = np.tile([atol_t, atol_c], n + 1)
224             step_observed, step_tail = (2.5, 150.0) if tight else (5.0, 300.0)
225             t, crossing = 0.0, None
226             work = dict(nfev=0, njev=0, nlu=0, accepted_steps=0, segments=0)

```

程序 2 code/q23_model.py (续)

```

221
222     def event(_t, state):
223         return float(np.max(state[1::2]) - THRESHOLD)
224
225     event.direction, event.terminal = -1, True
226     try:
227         while True:
228             if recorder.n_end is not None:
229                 target = max(recorder.n_end, common_end_s or 0, OBSERVED_END)
230                 if t >= target:
231                     break
232             elif crossing is not None:
233                 target = max(math.ceil(crossing), math.floor(t) + 1)
234             else:
235                 target = max_time_s
236             if t >= max_time_s:
237                 raise RuntimeError("Work/Excel-row limit reached before completing
the strict endpoint; not a drying-time answer")
238             if t < OBSERVED_END:
239                 end = min((math.floor(t / 60) + 1) * 60, target, max_time_s)
240                 system.boundary = boundary
241                 max_step = step_observed
242             else:
243                 end = min(t + 3600, target, max_time_s)
244                 system.boundary = boundary.right_side()
245                 max_step = step_tail
246             if end <= t:
247                 raise RuntimeError("Integration interval failed to advance")
248             sol = solve_ivp(system.rhs, (t, end), y, method="BDF", jac=system.jac,
249                             rtol=rtol, atol=atol, max_step=max_step,
250                             first_step=min(1e-3, end - t) if t == 0 else None,
251                             dense_output=True, events=event if crossing is None
else None)
252             for key in ("nfev", "njev", "nlu"):
253                 work[key] += int(getattr(sol, key))
254             work["segments"] += 1
255             work["accepted_steps"] += len(sol.t) - 1
256             if not sol.success:
257                 raise RuntimeError(f"BDF failed after t={t}: {sol.message}")
258             recorder.inspect(sol.y)
259             stop = float(sol.t[-1])
260             last_integer = math.floor(stop)
261             for first in range(recorder.last_time + 1, last_integer + 1,
SAMPLE_BLOCK):
262                 samples = np.arange(first, min(first + SAMPLE_BLOCK, last_integer
+ 1), dtype=np.int64)
263                 recorder.append(samples, sol.sol(samples))
264             if crossing is None and len(sol.t_events[0]):
265                 crossing = float(sol.t_events[0][0])

```

程序 2 code/q23_model.py (续)

```

266         recorder.snapshot(crossing, sol.y_events[0][0], "
threshold_crossing")
267         print(f"Threshold crossing: t={crossing:.9f} s, N={n}", flush=True
)
268         t, y = stop, sol.y[:, -1].copy()
269         if work["segments"] % 60 == 0 or crossing is not None:
270             print(f"N={n} {boundary.scenario}: t={t:.3f} s, max C={np.max(y
[1::2]):.9g}", flush=True)
271         if crossing is None or recorder.n_end is None or recorder.last_time != int
(t):
272             raise RuntimeError("Incomplete event or integer endpoint contract")
273         if recorder.n_end > MAX_EXCEL_DATA_ROWS:
274             raise RuntimeError("Dense sampled trajectory exceeds the configured
sample limit")
275         payload, roles = recorder.finish()
276         diagnostics = dict(success=True, status="computed",
277                             grid_n=int(n), scenario=boundary.scenario, tight=bool(
tight),
278                             t_cross_s=crossing, n_end=recorder.n_end,
279                             common_end_s=recorder.last_time, requested_common_end_s
=common_end_s,
280                             rtol=rtol, atol_temperature=atol_t, atol_moisture=
atol_c,
281                             max_step_observed_s=step_observed, max_step_tail_s=
step_tail,
282                             explicit_initial_step_s=1e-3,
283                             output_sample_block_size=SAMPLE_BLOCK, tail_segment_s
=3600,
284                             work_limit_s=max_time_s, threshold_kg_kg=THRESHOLD,
285                             snapshot_roles=roles, observed_extrema=recorder.extrema
,
286                             extrema_scope="all nodes at accepted steps and integer-
second dense samples",
287                             invariant_bounds=recorder.bounds, invariant_allowances=
recorder.allowances,
288                             work=work, elapsed_s=time.perf_counter() - started)
289         return payload, diagnostics
290     except Exception as exc:
291         exc.q2_diagnostics = dict(success=False, status="failed", grid_n=int(n),
292                                   scenario=boundary.scenario, tight=bool(tight),
293                                   failure_type=type(exc).__name__, failure_message=
str(exc),
294                                   last_complete_segment_time_s=t,
last_output_time_s=recorder.last_time,
295                                   t_cross_s=crossing, n_end=recorder.n_end,
296                                   observed_extrema=recorder.extrema, work=work,
297                                   elapsed_s=time.perf_counter() - started)
298         raise
299
300

```

程序 2 code/q23_model.py (续)

```
301 def run(n, output, data, *, tight=False, scenario="mean_tail"):
302     boundary=Boundary(Path(data)/"附件1.xlsx",scenario)
303     payload,metadata=solve(n,boundary,tight=tight)
304     return save_result(output,"q23",payload,metadata)
```

B.3 问题四：收缩域的热质耦合计算

在材料坐标上处理实测半径收缩，按题给附录 4 计算物性，并区分固定物理位置与实际表面。

程序 3 code/q4_model.py

```
1  """Q4: local coupled diffusion on an observed shrinking material domain."""
2  from __future__ import annotations
3  import math
4  import time
5  from pathlib import Path
6  import numpy as np
7  from scipy.integrate import solve_ivp
8  from scipy.sparse import coo_matrix
9  from common import Boundary, RadiusInput, save_result
10
11  R,H,HM=0.02,25.0,8e-7
12  INITIAL_T,INITIAL_C,THRESHOLD=28.0,2.55,0.15
13  OBSERVED_END,RADIUS_OBSERVED_END=14400,259200
14  SAMPLE_INTERVAL,SUMMARY_INTERVAL,SAMPLE_BLOCK=60,21600,64
15  MAX_MODEL_TIME=RADIUS_OBSERVED_END
16  MAX_EXCEL_DATA_ROWS=1048575
17
18  class System:
19      """Interleaved (theta_0,C_0,...,theta_N,C_N), with no diagnostic states."""
20
21      def __init__(self, n, boundary, radius, properties="appendix4"):
22          if isinstance(n, (bool, np.bool_)) or not isinstance(n, (int, np.integer))
23          or n < 1:
24              raise ValueError("N must be a positive integer")
25          if properties not in ("appendix3", "appendix4"):
26              raise ValueError("Unknown empirical-property appendix")
27          self.n, self.boundary, self.radius = int(n), boundary, radius
28          self.properties = properties
29          self.xi = np.linspace(0.0, 1.0, self.n + 1)
30          self.dx = 1.0 / self.n
31          faces = (self.xi[:-1] + self.xi[1:]) / 2
32          self.v = np.diff(np.r_[0.0, faces, 1.0] ** 2) / 2
33          self.q = faces / self.dx
34          self.h, self.hm = H, HM
35          self.size = 2 * (self.n + 1)
36
37      def split(self, y):
```

程序3 code/q4_model.py (续)

```

37     y = np.asarray(y)
38     if y.shape != (self.size,) or not np.isfinite(y).all():
39         raise ValueError("Invalid joint-state shape or nonfinite value")
40     theta, c = y[0::2], y[1::2]
41     # Real-part checks permit an infinitesimal complex-step oracle without
42     # modifying the analytic formulas. Production integration uses float64.
43     if np.any(c.real <= 0) or np.any(theta.real <= -273.15):
44         raise ValueError("Nonpositive moisture or absolute temperature; no
45 clipping")
46     return theta, c
47
48 def r_at(self, t):
49     return self.radius.evaluate(t) * self.xi
50
51 def coefficients(self, theta, c):
52     if self.properties == "appendix4":
53         rho0, rho1, cp0, cp1, k0, k1, d0, beta = 760., 90., 1850., 2150., .12,
54 .20, 4.2e-4, .30
55     else:
56         rho0, rho1, cp0, cp1, k0, k1, d0, beta = 650., 128., 1450., 2736.,
57 .21, .38, 2.4e-3, .45
58     rho = rho0 + rho1 * c
59     cp = cp0 + cp1 * c / (1.0 + c)
60     k = k0 + k1 * c / (1.0 + c)
61     d = d0 * np.exp(-beta / c - 3850.0 / (theta + 273.15))
62     return dict(rho=rho, cp=cp, s=rho * cp, k=k, D=d,
63                 s_C=rho1 * cp + rho * cp1 / (1.0 + c)**2,
64                 k_C=k1 / (1.0 + c)**2, D_C=beta * d / c**2,
65                 D_T=3850.0 * d / (theta + 273.15)**2)
66
67 def powers(self, t, theta, c, p):
68     radius = self.radius.evaluate(t)
69     dtype = np.result_type(theta, c, float)
70     ft = np.empty(self.n + 2, dtype=dtype)
71     fc = np.empty(self.n + 2, dtype=dtype)
72     ft[0] = fc[0] = 0
73     ft[1:-1] = self.q / radius**2 * (p["k"][:-1] + p["k"][1:]) / 2 * np.diff(
74 theta)
75     fc[1:-1] = self.q / radius**2 * (p["D"][:-1] + p["D"][1:]) / 2 * np.diff(
76 c)
77     t_air, c_eq = self.boundary.evaluate(t)
78     ft[-1] = self.h / radius * (t_air - theta[-1])
79     fc[-1] = self.hm / radius * (c_eq - c[-1])
80     return np.diff(ft), np.diff(fc)
81
82 def rhs(self, t, y):
83     theta, c = self.split(y)
84     p = self.coefficients(theta, c)
85     qt, qc = self.powers(t, theta, c, p)
86     out = np.empty(self.size, dtype=np.result_type(y, float))

```

程序 3 code/q4_model.py (续)

```

82     out[0::2] = qt / (self.v * p["s"])
83     out[1::2] = qc / self.v
84     return out
85
86     def jac(self, t, y):
87         radius = self.radius.evaluate(t)
88         theta, c = self.split(y)
89         p = self.coefficients(theta, c)
90         dt, dc = np.diff(theta), np.diff(c)
91         kb = (p["k"][:-1] + p["k"][1:]) / 2
92         db = (p["D"][:-1] + p["D"][1:]) / 2
93         # Each column is the derivative with respect to theta_i, C_i, theta_j, C_j.
94         jt = self.q[:, None] / radius**2 * np.column_stack(
95             (-kb, p["k_C"][:-1] * dt / 2, kb, p["k_C"][1:] * dt / 2))
96         jc = self.q[:, None] / radius**2 * np.column_stack(
97             (p["D_T"][:-1] * dc / 2, p["D_C"][:-1] * dc / 2 - db,
98              p["D_T"][1:] * dc / 2, p["D_C"][1:] * dc / 2 + db))
99         i = np.arange(self.n)
100        cols_face = 2 * i[:, None] + np.arange(4)[None, :]
101        rows, cols, vals = [], [], []
102        for offset, face_jac, weights in ((0, jt, self.v * p["s"]), (1, jc, self.v
103        ))):
104            for shift, sign in ((0, 1), (1, -1)):
105                nodes = i + shift
106                rows.append(np.repeat(2 * nodes + offset, 4))
107                cols.append(cols_face.ravel())
108                vals.append((sign * face_jac / weights[nodes, None]).ravel())
109            qt, _ = self.powers(t, theta, c, p)
110            theta_dot = qt / (self.v * p["s"])
111            nodes = np.arange(self.n + 1)
112            rows.extend([2 * nodes, np.array([2 * self.n, 2 * self.n + 1])])
113            cols.extend([2 * nodes + 1, np.array([2 * self.n, 2 * self.n + 1])])
114            vals.extend([-p["s_C"] / p["s"] * theta_dot,
115                        np.array([-self.h / radius / (self.v[-1] * p["s"][-1]),
116                                -self.hm / radius / self.v[-1]])])
117            return coo_matrix((np.concatenate(vals), (np.concatenate(rows), np.
118            concatenate(cols))),
119                             shape=(self.size, self.size)).tocsc()
120
121    class Recorder:
122        """Minute physical-radius samples plus complete material-grid key states."""
123
124        def __init__(self, system, boundary):
125            self.system = system
126            self.output_r = np.arange(21, dtype=float) * 0.001
127            self.samples, self.snapshots, self.roles = {}, {}, {}
128            self.last_regular_time = -SAMPLE_INTERVAL
129            self.n_end = None
130            self.endpoint_check = None

```

程序 3 code/q4_model.py (续)

```

130     self.extrema = dict(temperature_min_C=float("inf"), temperature_max_C=-
float("inf"),
131                          moisture_min_kg_kg=float("inf"), moisture_max_kg_kg=-
float("inf"))
132     self.bounds = dict(temperature_min_C=min(INITIAL_T, float(boundary.
temperature.min()), boundary.tail[0]),
133                       temperature_max_C=max(INITIAL_T, float(boundary.
temperature.max()), boundary.tail[0]),
134                       moisture_min_kg_kg=min(INITIAL_C, float(boundary.
moisture.min()), boundary.tail[1]),
135                       moisture_max_kg_kg=max(INITIAL_C, float(boundary.
moisture.max()), boundary.tail[1]))
136     self.allowances = dict(temperature_C=1e-7, moisture_kg_kg=1e-8)
137
138     def inspect(self, states):
139         a = np.asarray(states)
140         if not np.isfinite(a).all() or np.any(a[1::2] <= 0) or np.any(a[0::2] <=
-273.15):
141             raise RuntimeError("Accepted solution contains nonfinite/nonphysical
values")
142             for field, values, allowance in (("temperature", a[0::2], self.allowances[
"temperature_C"]),
143                                             ("moisture", a[1::2], self.allowances[
moisture_kg_kg"])):
144                 suffix = "C" if field == "temperature" else "kg_kg"
145                 low, high = f"{field}_min_{suffix}", f"{field}_max_{suffix}"
146                 self.extrema[low] = min(self.extrema[low], float(values.min()))
147                 self.extrema[high] = max(self.extrema[high], float(values.max()))
148                 if values.min() < self.bounds[low] - allowance or values.max() > self.
bounds[high] + allowance:
149                     raise RuntimeError(f"Accepted {field} exceeds the invariant-range
allowance")
150
151     def snapshot(self, t, y, role):
152         self.inspect(y)
153         self.snapshots[float(t)] = np.asarray(y, dtype=float).copy()
154         self.roles[role] = float(t)
155
156     def project(self, t, y):
157         """Interpolate only points inside the current material; surface is
separate."""
158         self.inspect(y)
159         theta, c = np.asarray(y)[0::2], np.asarray(y)[1::2]
160         rad = self.system.radius.evaluate(t)
161         # This tolerance only handles binary arithmetic equality at a surface.
162         eps = 16 * np.finfo(float).eps * max(R, rad)
163         mask = self.output_r <= rad + eps
164         x = np.minimum(self.output_r[mask] / rad, 1.0)
165         out_t, out_c = np.full(21, np.nan), np.full(21, np.nan)
166         out_t[mask] = np.interp(x, self.system.xi, theta)

```

程序3 code/q4_model.py (续)

```

167     out_c[mask] = np.interp(x, self.system.xi, c)
168     argmax = int(np.argmax(c))
169     return dict(temperature=out_t, moisture=out_c, inside=mask,
170                 surface_radius=rad, surface_temperature=float(theta[-1]),
171                 surface_moisture=float(c[-1]), maximum=float(c[argmax]),
172                 argmax=argmax, max_radius=float(rad * self.system.xi[argmax]),
173                 temperature_mean=float(2 * (self.system.v @ theta)),
174                 moisture_mean=float(2 * (self.system.v @ c)))
175
176     def append_regular(self, times, states):
177         times = np.asarray(times, dtype=np.int64)
178         if times.size == 0:
179             return
180         expected = self.last_regular_time + SAMPLE_INTERVAL * np.arange(1, len(
times) + 1)
181         if not np.array_equal(times, expected):
182             raise RuntimeError("Missing, repeated, or unordered 60-second sample")
183         if states.shape != (self.system.size, len(times)):
184             raise ValueError("Invalid regular-sample state array")
185         self.inspect(states)
186         for j, t in enumerate(times):
187             y = states[:, j]
188             self.samples[int(t)] = self.project(t, y)
189             if t == 0:
190                 self.snapshot(t, y, "initial")
191             if t == OBSERVED_END:
192                 self.snapshot(t, y, "environment_observed_end")
193             if t == RADIUS_OBSERVED_END:
194                 self.snapshot(t, y, "radius_observed_end")
195             if t > 0 and t % SUMMARY_INTERVAL == 0:
196                 self.snapshot(t, y, f"summary_{t}s")
197             self.last_regular_time = int(times[-1])
198
199     def accept_endpoint(self, t, y, preceding):
200         if t < 1 or np.max(y[1::2]) >= THRESHOLD or np.max(preceding[1::2]) <
THRESHOLD:
201             raise RuntimeError("Cannot establish the first strict integer-second
endpoint")
202         self.n_end = int(t)
203         before_node, after_node = int(np.argmax(preceding[1::2])), int(np.argmax(y
[1::2]))
204         self.endpoint_check = dict(
205             before_time_s=int(t - 1), before_max_moisture_kg_kg=float(preceding
[1::2][before_node]),
206             before_max_node=before_node, before_max_radius_m=float(self.system.
r_at(t - 1)[before_node]),
207             end_time_s=int(t), end_max_moisture_kg_kg=float(y[1::2][after_node]),
208             end_max_node=after_node, end_max_radius_m=float(self.system.r_at(t)[
after_node]),

```

程序3 code/q4_model.py (续)

```

209         scope="unrounded all-node values of this numerical run; cross-grid
integer stability is not asserted")
210         self.snapshot(t - 1, preceding, "before_n_end")
211         self.snapshot(t, y, "n_end")
212         self.samples[int(t)] = self.project(t, y)
213
214     def finish(self, end, y):
215         if int(end) != end:
216             raise RuntimeError("Final record requires an integer common endpoint")
217         self.snapshot(end, y, "common_end" if self.n_end is not None else "
horizon_end")
218         self.samples[int(end)] = self.project(end, y)
219         times = np.asarray(sorted(self.samples), dtype=np.int64)
220         entries = [self.samples[int(t)] for t in times]
221         snapshot_times = sorted(self.snapshots)
222         snapshots = np.asarray([self.snapshots[t] for t in snapshot_times])
223         def field(name, dtype=float):
224             return np.asarray([e[name] for e in entries], dtype=dtype)
225         payload = dict(
226             time_s=times, radius_m=self.output_r.copy(), mesh_xi=self.system.xi.
copy(),
227             temperature_C=field("temperature"), moisture_kg_kg=field("moisture"),
228             inside_mask=field("inside", bool), surface_radius_m=field("
surface_radius"),
229             surface_temperature_C=field("surface_temperature"),
surface_moisture_kg_kg=field("surface_moisture"),
230             max_moisture_kg_kg=field("maximum"), max_moisture_node=field("argmax",
np.int64),
231             max_moisture_radius_m=field("max_radius"), temperature_mean=field("
temperature_mean"),
232             moisture_mean=field("moisture_mean"),
233             official_output_mask=((times > 0) & (times <= self.n_end) &
234                 ((times % SAMPLE_INTERVAL == 0) | (times == self
.n_end))) if self.n_end is not None
235                 else np.zeros(times.shape, dtype=bool),
236             snapshot_time_s=np.asarray(snapshot_times, dtype=float),
237             snapshot_radius_m=self.system.radius.evaluate(np.asarray(
snapshot_times)),
238             temperature_snapshots=snapshots[:, 0::2], moisture_snapshots=snapshots
[:, 1::2])
239         roles = {role: snapshot_times.index(t) for role, t in self.roles.items()}
240         return payload, roles
241
242
243     def solve(n, boundary, radius, *, properties="appendix4", tight=False,
244             common_end_s=None, max_time_s=MAX_MODEL_TIME):
245         """Return unrounded data and diagnostics, without declaring independent
acceptance.
246
247         Event localization uses max over ALL material-grid nodes. Only minute output

```

程序3 code/q4_model.py (续)

```

248     samples and endpoint-neighbor probes are projected/stored, never every full
249     grid at every second. Accepted BDF states and all dense samples are inspected.
250     """
251     if isinstance(n, (bool, np.bool_)) or not isinstance(n, (int, np.integer)) or
n < 20 or n % 20:
252         raise ValueError("Production N must be a positive multiple of 20")
253     for name, value in (("common_end_s", common_end_s), ("max_time_s", max_time_s)
):
254         if value is not None and (isinstance(value, bool) or not isinstance(value,
(int, np.integer)) or value < 1):
255             raise ValueError(f"{name} must be a positive integer")
256         if max_time_s < OBSERVED_END or (common_end_s is not None and common_end_s >
max_time_s):
257             raise ValueError("Work ceiling cannot cover the requested comparison
interval")
258         if max(float(boundary.moisture.max()), boundary.tail[1]) >= THRESHOLD:
259             raise ValueError("This first-crossing endpoint contract requires all g(t)
< the threshold")
260     started = time.perf_counter()
261     system = System(n, boundary, radius, properties)
262     recorder = Recorder(system, boundary)
263     y0 = np.tile([INITIAL_T, INITIAL_C], n + 1)
264     y = y0.copy()
265     recorder.append_regular([0], y[:, None])
266     factor = 0.1 if tight else 1.0
267     rtol, atol_t, atol_c = 2e-9 * factor, 2e-9 * factor, 2e-11 * factor
268     atol = np.tile([atol_t, atol_c], n + 1)
269     step_observed, step_tail = (2.5, 150.0) if tight else (5.0, 300.0)
270     knots = np.unique(np.r_[boundary.times[1:], radius.breakpoints_s])
271     t, crossing = 0.0, None
272     previous_dense = None
273     horizon_stop_reason = None
274     next_candidate, preceding_candidate = None, None
275     work = dict(nfev=0, njev=0, nlu=0, accepted_steps=0, segments=0)
276
277     def event(_t, state):
278         return float(np.max(state[1::2]) - THRESHOLD)
279     event.direction, event.terminal = -1, True
280
281     try:
282         while True:
283             if recorder.n_end is not None:
284                 target = max(recorder.n_end, common_end_s or 0, math.ceil(t))
285                 if t >= target:
286                     break
287             elif crossing is not None:
288                 target = max(next_candidate, math.floor(t) + 1)
289             else:
290                 target = max_time_s
291             if t >= max_time_s:

```

程序3 code/q4_model.py (续)

```

292         horizon_stop_reason = "explicit_model_time_limit"
293         break
294         if (radius.method != "constant" and radius.tail == "error" and t >=
RADIUS_OBSERVED_END):
295             horizon_stop_reason = "
radius_observation_limit_without_continuation"
296             break
297             ix = int(np.searchsorted(knots, t, side="right"))
298             next_knot = float(knots[ix]) if ix < len(knots) else float("inf")
299             end = min(next_knot, t + 3600, target, max_time_s)
300             if radius.method != "constant" and radius.tail == "error":
301                 end = min(end, RADIUS_OBSERVED_END)
302             if end <= t:
303                 raise RuntimeError("Integration interval failed to advance")
304             system.boundary = boundary if t < OBSERVED_END else boundary.
right_side()
305             max_step = step_observed if t < OBSERVED_END else step_tail
306             start = t
307             sol = solve_ivp(system.rhs, (start, end), y, method="BDF", jac=system.
jac,
308                             rtol=rtol, atol=atol, max_step=max_step,
309                             first_step=min(1e-3, end - start) if start == 0 else
None,
310                             dense_output=True, events=event if crossing is None
else None)
311             for key in ("nfev", "njev", "nlu"):
312                 work[key] += int(getattr(sol, key))
313             work["segments"] += 1
314             work["accepted_steps"] += len(sol.t) - 1
315             if not sol.success:
316                 raise RuntimeError(f"BDF failed after t={start}: {sol.message}")
317             recorder.inspect(sol.y)
318             stop = float(sol.t[-1])
319             last_regular = SAMPLE_INTERVAL * math.floor(stop / SAMPLE_INTERVAL)
320             for first in range(recorder.last_regular_time + SAMPLE_INTERVAL,
last_regular + 1,
321                             SAMPLE_INTERVAL * SAMPLE_BLOCK):
322                 sample_times = np.arange(first, min(first + SAMPLE_INTERVAL *
SAMPLE_BLOCK,
323                                                     last_regular + SAMPLE_INTERVAL
), SAMPLE_INTERVAL,
324                                         dtype=np.int64)
325                 recorder.append_regular(sample_times, sol.sol(sample_times))
326
327             def state_at_when_available(instant):
328                 # Never extrapolate a dense polynomial across an input knot.
329                 if start <= instant <= stop:
330                     value = np.asarray(sol.sol(instant), dtype=float)
331                     elif previous_dense is not None and previous_dense[0] <= instant
<= previous_dense[1]:

```

程序 3 code/q4_model.py (续)

```

332         value = np.asarray(previous_dense[2](instant), dtype=float)
333     elif instant == 0:
334         value = y0.copy()
335     else:
336         raise RuntimeError("Required endpoint-neighbor state lies
outside retained dense intervals")
337     recorder.inspect(value)
338     return value
339
340     new_crossing = crossing is None and len(sol.t_events[0]) > 0
341     if new_crossing:
342         crossing = float(sol.t_events[0][0])
343         recorder.snapshot(crossing, sol.y_events[0][0], "
threshold_crossing")
344         next_candidate = max(1, math.floor(crossing))
345         preceding_candidate = state_at_when_available(next_candidate - 1)
346         if np.max(preceding_candidate[1::2]) < THRESHOLD:
347             raise RuntimeError("Numerical event follows an already strict
preceding integer; refine the event check")
348             print(f"Threshold crossing: t={crossing:.9f} s, N={n}", flush=True
)
349         if crossing is not None and recorder.n_end is None:
350             while next_candidate <= math.floor(stop):
351                 candidate_state = state_at_when_available(next_candidate)
352                 if np.max(candidate_state[1::2]) < THRESHOLD:
353                     recorder.accept_endpoint(next_candidate, candidate_state,
preceding_candidate)
354                     break
355                 preceding_candidate = candidate_state
356                 next_candidate += 1
357         # This view keeps only one previous interval; older dense coefficients
358         # are released. It supports a crossing exactly at an input knot.
359         previous_dense = (start, stop, sol.sol)
360         t, y = stop, sol.y[:, -1].copy()
361         if work["segments"] % 60 == 0 or new_crossing:
362             print(f"Q4 N={n} {boundary.scenario}/{radius.method}/{properties}:
"
363
364                 f"t={t:.3f} s, R={radius.evaluate(t):.8f} m, max C={np.max(y
[1::2]):.9g}", flush=True)
364             if int(t) != t:
365                 raise RuntimeError("Final modeled horizon is not an integer second")
366             drying_complete = crossing is not None and recorder.n_end is not None
367             comparison_complete = common_end_s is None or t >= common_end_s
368             complete = drying_complete and comparison_complete
369             expected_minutes = np.arange(0, SAMPLE_INTERVAL * math.floor(t /
SAMPLE_INTERVAL) + 1,
370
371                                     SAMPLE_INTERVAL, dtype=np.int64)
371             if any(int(instant) not in recorder.samples for instant in
expected_minutes):
372                 raise RuntimeError("Missing a required minute sample")

```

程序3 code/q4_model.py (续)

```

373     payload, roles = recorder.finish(t, y)
374     if not complete:
375         payload["official_output_mask"][:] = False
376     if int(payload["official_output_mask"].sum()) > MAX_EXCEL_DATA_ROWS:
377         raise RuntimeError("The result4 time table exceeds one Excel worksheet
")
378     diagnostics = dict(success=bool(complete),
379                        status=("computed" if complete
380                               else "threshold_not_reached_within_horizon" if
not drying_complete
381                               else "comparison_horizon_not_completed"),
382                        drying_complete=bool(drying_complete),
comparison_complete=bool(comparison_complete),
383                        horizon_stop_reason=horizon_stop_reason,
384                        final_max_moisture_kg_kg=float(np.max(y[1::2])),
385                        final_max_moisture_radius_m=float(system.r_at(t)[np.
argmax(y[1::2])]),
386                        grid_n=int(n), scenario=boundary.scenario, properties=
properties,
387                        initial_temperature_C=INITIAL_T, initial_moisture_kg_kg
=INITIAL_C,
388                        initial_radius_m=float(radius.evaluate(0)),
389                        radius=radius.describe(t), tight=bool(tight),
390                        t_cross_s=crossing, n_end=recorder.n_end, common_end_s=
int(t),
391                        requested_common_end_s=common_end_s,
392                        crossing_within_radius_observations=bool(crossing <=
RADIUS_OBSERVED_END) if crossing is not None else None,
393                        strict_endpoint_within_radius_observations=bool(
recorder.n_end <= RADIUS_OBSERVED_END) if recorder.n_end is not None else None,
394                        rtol=rtol, atol_temperature=atol_t, atol_moisture=
atol_c,
395                        max_step_observed_s=step_observed, max_step_tail_s=
step_tail,
396                        explicit_initial_step_s=1e-3, output_interval_s=
SAMPLE_INTERVAL,
397                        output_sample_block_size=SAMPLE_BLOCK,
tail_segment_max_s=3600,
398                        work_limit_s=max_time_s, threshold_kg_kg=THRESHOLD,
399                        snapshot_roles=roles, endpoint_check=recorder.
endpoint_check,
400                        observed_extrema=recorder.extrema,
401                        extrema_scope="all material nodes at accepted BDF steps
, 60-second dense samples, "
402                                "event/neighbor probes, and selected
complete snapshots; not a continuum bound",
403                        invariant_bounds=recorder.bounds, invariant_allowances=
recorder.allowances,
404                        output_domain="fixed physical radii; NaN with
inside_mask=false outside the material; "

```

程序 3 code/q4_model.py (续)

```

405         "moving surface is an independent column"
406     ,
407         work=work, elapsed_s=time.perf_counter() - started)
408     return payload, diagnostics
409 except Exception as exc:
410     exc.q4_diagnostics = dict(success=False, status="failed", grid_n=int(n),
411                               scenario=boundary.scenario, properties=properties
412     ,
413         radius=radius.describe(t), tight=bool(tight),
414         failure_type=type(exc).__name__, failure_message=
415         str(exc),
416         last_complete_segment_time_s=t,
417         last_regular_output_time_s=recorder.
418         last_regular_time,
419         t_cross_s=crossing, n_end=recorder.n_end,
420         observed_extrema=recorder.extrema, work=work,
421         elapsed_s=time.perf_counter() - started)
422     raise
423
424 def run(n, output, data, *, tight=False, scenario="mean_tail", radius_method="
425 linear"):
426     boundary=Boundary(Path(data)/"附件1.xlsx",scenario)
427     radius=RadiusInput(Path(data)/"附件2.xlsx",method=radius_method)
428     payload,metadata=solve(n,boundary,radius,tight=tight)
429     if not metadata["success"]:
430         raise RuntimeError("Drying criterion not reached within the radius
431 observations")
432     return save_result(output,"q4",payload,metadata)

```

B.4 数据接口与公共参数

读取题给环境与半径观测，构造分段插值和长期环境延续接口。

程序 4 code/common.py

```

1  """Raw workbook inputs, units and portable numerical-result storage."""
2  from __future__ import annotations
3
4  import json
5  from decimal import Decimal
6  from pathlib import Path
7  import numpy as np
8  import scipy
9  import openpyxl
10 from scipy.interpolate import PchipInterpolator
11
12 DATA = Path(__file__).resolve().parents[1] / "data"
13

```

程序4 code/common.py (续)

```

14
15 def numeric_rows(path, columns):
16     book = openpyxl.load_workbook(path, read_only=True, data_only=True)
17     try:
18         rows = list(book.worksheets[0].iter_rows(min_row=2, max_col=columns,
19 values_only=True))
19         values = np.asarray(rows, dtype=float)
20     finally:
21         book.close()
22     if values.ndim != 2 or values.shape[1] != columns or not np.isfinite(values).
23 all():
24         raise ValueError(f"Invalid numeric input: {Path(path).name}")
25     return values
26
27 class Boundary:
28     """Linear observed input; after four hours use the selected constant tail."""
29     def __init__(self, path=DATA / "附件1.xlsx", scenario="mean_tail", *, q1=False
30 ):
31         if scenario not in ("mean_tail", "last_value"):
32             raise ValueError("Unknown boundary scenario")
33         a = numeric_rows(path, 3)
34         if a.shape != (241, 3) or not np.array_equal(a[:, 0], np.arange(0, 14401,
35 60)):
36             raise ValueError("Expected 241 observations at 60-second intervals")
37         if np.any(a[:, 1] <= -273.15) or np.any(a[:, 2] <= 0):
38             raise ValueError("Invalid environmental temperature or moisture")
39         self.scenario, self.q1 = scenario, q1
40         # Trapezoidal integration of the final hour's linear interpolant.
41         tail_values = a[180:241, 1:]
42         mean = [float(sum((Decimal(str(x)) + Decimal(str(y))) * 30
43 for x, y in zip(tail_values[:-1, j], tail_values[1:, j])
44 )
45 / Decimal(3600)) for j in range(2)]
46         self.tail = tuple(map(float, mean if scenario == "mean_tail" else a[-1,
47 1:]))
48         a = a[:31] if q1 else a
49         self.times, self.temperature, self.moisture = (a[:, j].copy() for j in
50 range(3))
51
52 def evaluate(self, t):
53     if not np.isscalar(t) or not np.isfinite(t) or t < 0:
54         raise ValueError("Boundary time must be a finite nonnegative scalar")
55     if self.q1 and t > 1800:
56         raise ValueError("Q1 input is limited to 0--1800 seconds")
57     if t <= self.times[-1]:
58         return (float(np.interp(t, self.times, self.temperature)),
59 float(np.interp(t, self.times, self.moisture)))
60     return self.tail

```

程序4 code/common.py (续)

```

57     def right_side(self):
58         tail = self.tail
59         class RightSide:
60             def evaluate(self, t):
61                 if not np.isscalar(t) or not np.isfinite(t) or t < 14400:
62                     raise ValueError("Right-side boundary requires t >= 14400
seconds")
63             return tail
64         return RightSide()
65
66
67 class RadiusInput:
68     """Observed radius in metres; material equations do not require R-prime."""
69     def __init__(self, path=DATA / "附件2.xlsx", method="linear", *,
70                 tail="error", constant_radius_m=0.02):
71         if method not in ("linear", "pchip", "constant") or tail not in ("error",
"hold_last"):
72             raise ValueError("Unknown radius interpolation or continuation")
73         a = numeric_rows(path, 2)
74         if a.shape != (145, 2) or not np.array_equal(a[:, 0], np.arange(0, 259201,
1800)):
75             raise ValueError("Expected 145 radius observations at 1800-second
intervals")
76         self.times, self.radius_cm = a[:, 0].copy(), a[:, 1].copy()
77         if (not np.isfinite(constant_radius_m) or constant_radius_m <= 0
78             or np.any(self.radius_cm <= 0) or np.any(np.diff(self.radius_cm) >
0)
79             or self.radius_cm[0] != 2.0 or self.radius_cm[-1] != 1.198):
80             raise ValueError("Radius must be positive, nonincreasing and initially
0.02 m")
81         self.method, self.tail, self.constant_radius_m = method, tail, float(
constant_radius_m)
82         self.breakpoints_s = self.times[1:].copy() if method != "constant" else np
.empty(0)
83         self.pchip = PchipInterpolator(self.times, self.radius_cm, extrapolate=
False) if method == "pchip" else None
84
85     def evaluate(self, t):
86         a = np.asarray(t, dtype=float)
87         if not np.isfinite(a).all() or np.any(a < 0):
88             raise ValueError("Radius time must be finite and nonnegative")
89         if self.method == "constant":
90             values = np.full_like(a, self.constant_radius_m, dtype=float)
91         else:
92             if self.tail == "error" and np.any(a > self.times[-1]):
93                 raise ValueError("Radius requested beyond the observed interval")
94             observed_t = np.minimum(a, self.times[-1])
95             radius_cm = (np.interp(observed_t, self.times, self.radius_cm)
96                 if self.method == "linear" else self.pchip(observed_t))
97             values = radius_cm * 0.01

```

程序4 code/common.py (续)

```

98     if np.any(values <= 0) or not np.isfinite(values).all():
99         raise ValueError("Invalid evaluated radius")
100     return float(values) if np.ndim(values) == 0 else values
101
102     def describe(self, end_time):
103         return dict(method=self.method, observed_interval_s=[0, 259200],
104                     continuation=self.tail,
105                     continuation_used=bool(self.method != "constant" and end_time
106 > 259200),
107                     constant_radius_m=self.constant_radius_m if self.method == "
108 constant" else None,
109                     input_units="cm", internal_units="m", source_node_count=145)
110
111 def save_result(output_dir, name, payload, metadata):
112     """Save unrounded fields and settings; never replace an existing calculation."
113     """
114     output_dir = Path(output_dir).resolve()
115     output_dir.mkdir(parents=True, exist_ok=True)
116     result, settings = output_dir / f"{name}.npz", output_dir / f"{name}.json"
117     if result.exists() or settings.exists():
118         raise FileExistsError("Choose a new output directory; results are not
119 overwritten")
120     metadata = dict(metadata, numpy=np.__version__, scipy=scipy.__version__)
121     np.savez_compressed(result, **payload)
122     settings.write_text(json.dumps(metadata, ensure_ascii=False, indent=2) + "\n",
123                          encoding="utf-8")
124     return result

```

B.5 运行入口

选择问题、设置网格与容差，并限制单次计算时间。问题二与问题三复用同一条耦合计算轨迹，分别导出前三小时与完整干燥过程。

程序5 code/run_models.py

```

1  """Portable command-line runner with a total wall-clock limit."""
2  from __future__ import annotations
3  import argparse
4  import subprocess
5  import sys
6  import time
7  from pathlib import Path
8
9
10 def main():
11     parser = argparse.ArgumentParser(description=__doc__)
12     parser.add_argument("--case", choices=("q1", "q23", "q4", "all"), default="all
    ")

```

程序 5 code/run_models.py (续)

```

13     parser.add_argument("--output", type=Path, default=Path("recomputed"))
14     parser.add_argument("--data", type=Path, default=Path(__file__).resolve().
    parents[1] / "data")
15     parser.add_argument("--n", type=int, help="Radial intervals; default Q1=5120,
    Q23/Q4=10240")
16     parser.add_argument("--tight", action="store_true", help="Use the existing
    tighter time-error settings")
17     parser.add_argument("--tail", choices=("mean_tail", "last_value"), default="
    mean_tail")
18     parser.add_argument("--radius", choices=("linear", "pchip", "constant"),
    default="linear")
19     parser.add_argument("--timeout", type=float, default=3300.0, help="Total wall-
    clock budget in seconds, strictly below one hour")
20     parser.add_argument("--no-xlsx", action="store_true", help="Save unrounded NPZ
    /settings only")
21     parser.add_argument("--worker", action="store_true", help=argparse.SUPPRESS)
22     args = parser.parse_args()
23     if not 0 < args.timeout <= 3300:
24         parser.error("--timeout must be in (0,3300] seconds")
25     if args.n is not None and (args.n < 20 or args.n % 20):
26         parser.error("--n must be a positive multiple of 20")
27     cases = ("q1", "q23", "q4") if args.case == "all" else (args.case,)
28     if args.worker:
29         if args.case == "all":
30             parser.error("Worker accepts one calculation")
31         if args.case == "q1":
32             from q1_model import run
33             run(args.n or 5120, args.output, args.data, tight=args.tight)
34         elif args.case == "q23":
35             from q23_model import run
36             run(args.n or 10240, args.output, args.data, tight=args.tight,
    scenario=args.tail)
37         else:
38             from q4_model import run
39             run(args.n or 10240, args.output, args.data, tight=args.tight,
    scenario=args.tail, radius_method=args.radius)
40         if not args.no_xlsx:
41             from export_results import export_case
42             for path in export_case(args.case, args.output, args.data):
43                 print(path, flush=True)
44         return
45     # The entire command, including --case all, shares one wall-clock budget.
46     deadline = time.monotonic() + args.timeout
47     for case in cases:
48         command = [sys.executable, "-B", str(Path(__file__).resolve()), "--worker"
    , "--case", case,
50             "--output", str(args.output.resolve()), "--data", str(args.data
    .resolve()),
51             "--tail", args.tail, "--radius", args.radius, "--timeout", str(
    args.timeout)]

```

程序 5 code/run_models.py (续)

```

52     if args.n is not None:
53         command += ["--n", str(args.n)]
54     if args.tight:
55         command.append("--tight")
56     if args.no_xlsx:
57         command.append("--no-xlsx")
58     remaining = deadline - time.monotonic()
59     if remaining <= 0:
60         raise SystemExit("Total wall-clock budget exhausted; calculation
stopped")
61     try:
62         subprocess.run(command, check=True, timeout=remaining)
63     except subprocess.TimeoutExpired as exc:
64         raise SystemExit(f"{case}: exceeded {args.timeout:g} seconds;
calculation stopped") from exc
65
66
67 if __name__ == "__main__":
68     main()

```

B.6 结果工作簿导出

按题给模板写入规定时空点的四位小数结果；域外位置保留空白。

程序 6 code/export_results.py

```

1  """Write the four prescribed tables from unrounded numerical arrays."""
2  from __future__ import annotations
3  from copy import copy
4  import json
5  from pathlib import Path
6  import numpy as np
7  import openpyxl
8  from common import DATA
9
10
11 def four(value):
12     return round(float(value), 4) if np.isfinite(value) else None
13
14
15 def write_table(number, output_dir, times, fields, *, surface=None, data_dir=DATA)
:
16     """Expand the original ellipsis to every prescribed 0.1 cm position."""
17     output_dir = Path(output_dir)
18     target = output_dir / f"result{number}.xlsx"
19     if target.exists():
20         raise FileExistsError(f"Refusing to overwrite {target.name}")
21     book = openpyxl.load_workbook(Path(data_dir) / "templates" / target.name)
22     if len(book.worksheets) != len(fields):

```

程序 6 code/export_results.py (续)

```

23     raise ValueError("Template worksheet count differs from requested fields")
24     radii = [round(j / 10, 1) for j in range(21)]
25     for sheet, values in zip(book.worksheets, fields):
26         header, style = sheet.cell(1, 1).value, copy(sheet.cell(1, 2)._style)
27         if np.asarray(values).shape != (len(times), 21):
28             raise ValueError("Expected one value per time and physical radial
position")
29         sheet.delete_rows(1, sheet.max_row)
30         sheet.append([header, *radii] + (["药材表面"] if surface is not None else
[]))
31         for cell in sheet[1]:
32             cell._style = copy(style)
33         for j, (t, row) in enumerate(zip(times, values)):
34             sheet.append([int(t), *(four(v) for v in row)]
+ ([four(surface[j])] if surface is not None else []))
35         for row in sheet.iter_rows(min_row=2, min_col=2):
36             for cell in row:
37                 cell.number_format = "0.0000"
38             sheet.freeze_panes = "B2"
39             sheet.column_dimensions["A"].width = 27
40         book.save(target)
41         book.close()
42         return target
43
44
45
46 def export_case(case, output_dir, data_dir=DATA):
47     output_dir = Path(output_dir)
48     with np.load(output_dir / f"{case}.npz", allow_pickle=False) as a:
49         meta = json.loads((output_dir / f"{case}.json").read_text(encoding="utf-8"
))
50         times = a["time_s"]
51         if case == "q1":
52             use = (times >= 1) & (times <= 1800)
53             return [write_table(1, output_dir, times[use],
54                             [a["temperature_C"][use], a["moisture_kg_kg"][use
]], data_dir=data_dir)]
55         if case == "q23":
56             early = (times >= 1) & (times <= 10800)
57             files = [write_table(2, output_dir, times[early],
58                             [a["temperature_C"][early], a["moisture_kg_kg"][
early]], data_dir=data_dir)]
59             end = int(meta["n_end"])
60             use = (times > 0) & (times <= end) & ((times % 60 == 0) | (times ==
end))
61             files.append(write_table(3, output_dir, times[use], [a["moisture_kg_kg
"]][use]], data_dir=data_dir))
62             return files
63         if case == "q4":
64             use = a["official_output_mask"]
65             moisture = a["moisture_kg_kg"][use].copy()

```

程序 6 code/export_results.py (续)

```
66         moisture[~a["inside_mask"][use]] = np.nan
67         return [write_table(4, output_dir, times[use], [moisture],
68                           surface=a["surface_moisture_kg_kg"][use], data_dir
        =data_dir)]
69         raise ValueError("Unknown calculation")
```

B.7 绘图数值样本提取

读取已有计算结果，提取图件所需的数值数组及元数据。

程序 7 code/extract_figure_data.py

```
1  """Extract only plotted arrays from existing saved results (no model execution).
2
3  Use explicit input filenames. This extractor does not require the original
4  workspace, diagnostics directory, source hashes, or any solver import.
5  Measured radius metadata already supplied with the appendix is retained.
6  """
7  from pathlib import Path
8  import argparse
9  import hashlib
10 import json
11 import shutil
12 import subprocess
13 import sys
14 import numpy as np
15
16 def extract(q1, q2, q4, output, q4_end=None, observations=None):
17     output = Path(output); output.mkdir(parents=True, exist_ok=True)
18     inputs = {}
19     for key, path in [('q1', q1), ('q2', q2), ('q4', q4)]:
20         path = Path(path)
21         inputs[key] = {'filename': path.name, 'sha256': hashlib.sha256(path.
read_bytes()).hexdigest()}
22     def keep(name, **arrays):
23         np.savez_compressed(output/name, **arrays)
24     with np.load(q1, allow_pickle=False) as z:
25         times = np.array([100, 600, 900, 1200, 1800])
26         ids = np.array([np.flatnonzero(z['time_s']==t).item() for t in times])
27         keep('q1_profiles.npz', time_s=times, radius_m=z['radius_m'],
28             temperature_C=z['temperature_C'][ids], moisture_kg_kg=z['
moisture_kg_kg'][ids])
29     with np.load(q2, allow_pickle=False) as z:
30         hits=np.flatnonzero(z['max_moisture_kg_kg']<.15)
31         if not len(hits):
32             raise ValueError('Q2/Q3结果中尚未达到全域阈值，不能绘制完整干燥图。')
33         q3_end=int(z['time_s'][hits[0]])
34         t = z['time_s']; ids = np.flatnonzero((t>=0)&(t<=10800))
35         assert np.array_equal(t[ids], np.arange(10801))
```

程序7 code/extract_figure_data.py (续)

```

36     keep('q2_fields.npz', time_s=t[ids], radius_m=z['radius_m'],
37         temperature_C=z['temperature_C'][ids], moisture_kg_kg=z['
moisture_kg_kg'][ids])
38     with np.load(q4, allow_pickle=False) as z:
39         if q4_end is None:
40             hits=np.flatnonzero(z['max_moisture_kg_kg']<.15)
41             if not len(hits):
42                 raise ValueError('Q4结果中尚未达到全域阈值。')
43             q4_end=int(z['time_s'][hits[0]])
44             times = np.array([0, 21600, 64800, 129600, 172800, q4_end])
45             ids = np.array([np.flatnonzero(z['snapshot_time_s']==t).item() for t in
times])
46             keep('q4_sections.npz', mesh_xi=z['mesh_xi'], snapshot_time_s=times,
47                 snapshot_radius_m=z['snapshot_radius_m'][ids], moisture_snapshots=z['
moisture_snapshots'][ids])
48             observations=Path(observations) if observations else Path(__file__).resolve().
parents[1]/'figure_data/radius_observations.json'
49             if observations.is_file():
50                 observed=json.loads(observations.read_text(encoding='utf-8'))
51                 (output/'radius_observations.json').write_text(json.dumps(observed,
ensure_ascii=False,indent=2)+'\n',encoding='utf-8')
52                 subprocess.run([sys.executable, '-B', str(Path(__file__).with_name('
figure_restructured.py')),
53                     '--source', str(Path(q2).resolve()), '--data-dir', str(output.
resolve()),
54                     '--extract-only'], check=True, timeout=120)
55             files = {}
56             for path in sorted(output.glob('*.npz')):
57                 with np.load(path, allow_pickle=False) as z:
58                     files[path.name] = {'sha256': hashlib.sha256(path.read_bytes()).
hexdigest(),
59                                     'bytes': path.stat().st_size,
60                                     'arrays': {key: {'shape': list(z[key].shape), '
dtype': str(z[key].dtype)} for key in z.files}}
61             manifest = {'inputs': inputs, 'files': files, 'q3_end_s': q3_end, 'q4_end_s':
q4_end,
62                         'units': {'time_s': 's', 'radius_m': 'm', 'temperature_C': 'degC',
63                                 'moisture_kg_kg': 'kg/kg dry basis', 'mesh_xi': '
dimensionless'},
64                         'operation': 'Exact array selection; no rounding, fitting,
filtering, or solver execution.',
65                         'q3_maximum': 'Saved maximum over all numerical nodes. Completion
bands display the 21 prescribed radii.',
66                         'additional_data': 'restructured_figure_data.json contains exact
diffusivity decomposition and integer-second radial threshold crossings.'}
67             (output/'data_manifest.json').write_text(json.dumps(manifest, ensure_ascii=
False, indent=2)+'\n', encoding='utf-8')
68
69 def main():
70     p = argparse.ArgumentParser(description=__doc__)

```

程序7 code/extract_figure_data.py (续)

```
71     for name in ('q1', 'q2', 'q4', 'output'):
72         p.add_argument('--'+name, type=Path, required=True)
73         p.add_argument('--q4-end', type=int, help='Override first saved all-node
threshold time')
74         p.add_argument('--observations', type=Path, help='Optional replacement
measured-radius JSON')
75         a = p.parse_args()
76         extract(a.q1, a.q2, a.q4, a.output, a.q4_end, a.observations)
77
78 if __name__ == '__main__':
79     main()
```

B.8 实测收缩与材料坐标图

结合半径观测说明固定物理位置与材料坐标之间的关系。

程序8 code/figure_coordinates.py

```
1  """Portable functions extracted from the team's existing figure source.
2  No solver is imported. See figures/README.md for scope and data provenance.
3  """
4  from pathlib import Path
5  import json
6  import matplotlib
7  matplotlib.use("Agg")
8  import matplotlib.pyplot as plt
9  from matplotlib import font_manager
10 from matplotlib.colors import LinearSegmentedColormap, Normalize, to_rgb
11 from matplotlib.cm import ScalarMappable
12 from matplotlib.lines import Line2D
13 from matplotlib.patches import Circle
14 from matplotlib.text import Text
15 import matplotlib.patheffects as path_effects
16 import numpy as np
17 DATA = Path(__file__).resolve().parents[1] / "figure_data"
18 OUT = Path.cwd() / "redrawn_figures"
19
20 def obs():
21     return json.loads((DATA/"radius_observations.json").read_text(encoding="utf-8"
22 ))
23
24 def export(fig, name, dpi=300, transparent=False):
25     OUT.mkdir(parents=True, exist_ok=True)
26     for ext in ("pdf", "svg", "png"):
27         fig.savefig(OUT/f"{name}.{ext}", dpi=dpi, transparent=transparent)
28     plt.close(fig)
29
30 WIDTH_CM, HEIGHT_CM=15.8, 6.4
31 BLUE, ORANGE, INK="#0072B2", "#D55E00", "#45525B"
```

程序8 code/figure_coordinates.py (续)

```

31 BODY,GRAY="#F3EBDD","#747474"
32 def gray(color):
33     rgb = np.array(to_rgb(color))
34     linear = np.where(rgb <= 0.04045, rgb / 12.92, ((rgb + 0.055) / 1.055) ** 2.4)
35     lum = float(linear @ np.array([0.2126, 0.7152, 0.0722]))
36     srgb = 12.92 * lum if lum <= 0.0031308 else 1.055 * lum ** (1 / 2.4) - 0.055
37     return (srgb, srgb, srgb)
38
39 def draw(times, radii, selected, *, grayscale=False):
40     color = gray if grayscale else lambda value: value
41     blue, orange, ink, body, neutral = map(color, (BLUE, ORANGE, INK, BODY, GRAY))
42     fig = plt.figure(figsize=(WIDTH_CM / 2.54, HEIGHT_CM / 2.54))
43     chart = fig.add_axes((0.086, 0.195, 0.325, 0.705))
44     chart.plot(times, radii, color=blue, lw=1.75, label="线性重建", zorder=2)
45     chart.scatter(times, radii, s=5.1, color=neutral, edgecolors="none", label="半
    径观测", zorder=3)
46     chart.set(xlim=(0, 72), ylim=(1.16, 2.035), xlabel="时间 / h")
47     chart.set_xticks([0, 18, 36, 54, 72])
48     chart.set_yticks([1.2, 1.4, 1.6, 1.8, 2.0])
49     chart.tick_params(labelsize=8, length=2.8, width=0.7, pad=3)
50     chart.grid(color="0.90", lw=0.45)
51     chart.set_axisbelow(True)
52     chart.legend(frameon=False, loc="upper right", bbox_to_anchor=(1.0, 0.84),
53                 fontsize=8, handlelength=2.1, handletextpad=0.45)
54     chart.text(-0.17, 0.48, "外\n半\n径\n(cm)", transform=chart.transAxes,
55              ha="center", va="center", fontsize=8.5, linespacing=1.05)
56     chart.text(0.0, 1.055, "(a)", transform=chart.transAxes, ha="left", va="bottom
    ", fontsize=9)
57
58     # Both snapshots live in a single equal-aspect axis, so their physical
59     # centimetre scales cannot diverge through independent autoscaling.
60     view = fig.add_axes((0.452, 0.10, 0.538, 0.80))
61     view.set(xlim=(-2.3, 8.45), ylim=(-3.0, 3.10))
62     view.set_aspect("equal", adjustable="box")
63     view.axis("off")
64     centers = [0.0, 5.75]
65     for center, radius, hour in zip(centers, selected, [0, 6]):
66         view.add_patch(Circle((center, 0), radius, facecolor=body, edgecolor=ink,
67                               lw=1.15, zorder=1))
68         view.add_patch(Circle((center, 0), radius * 0.5, fill=False, edgecolor=
69                               blue, lw=1.65, zorder=3))
70         view.plot([center - 2.1, center + 2.2], [0, 0], color="0.70", lw=0.65,
71                  ls=(0, (4, 2, 1, 2)), zorder=2)
72         view.plot(center, 0, "o", ms=2.7, color=ink, zorder=6)
73         # A tagged material point lies on the same xi=0.5 circle in each view.
74         view.plot(center + radius * 0.5, 0, "o", ms=4.3, mfc="white", mec=blue,
75                  mew=1.2, zorder=7)
76         # This is a fixed-coordinate sample marker, not a physical instrument.
77         fixed_x = center + 1.5
78         view.plot([fixed_x, fixed_x], [-0.37, 0.50], color=orange, lw=1.1,

```

程序 8 code/figure_coordinates.py (续)

```

76         ls=(0, (2.5, 2.0)), zorder=4)
77     view.plot(fixed_x, 0, marker="x", ms=4.3, mew=1.15, color=orange, zorder
78 =8)
79     view.text(center, 2.48, f"{hour} h", ha="center", va="center", fontsize
80 =8.8)
81     view.text(center, -2.49, rf"$R={radius:.3f}\,\mathrm{{cm}}$",
82             ha="center", va="center", fontsize=8.6)
83     view.text(center - 0.08, -0.27, "O", ha="right", va="top", fontsize=7.6)
84     # Sequence direction only; no arrow length encodes material velocity.
85     view.annotate("", xy=(3.49, 0.88), xytext=(2.50, 0.88),
86                 arrowprops=dict(arrowstyle="->", lw=1.15, color=ink))
87     view.text(2.99, 1.22, r"$r=R(t)\xi$", ha="center", va="bottom", fontsize=8.4)
88     fig.text(0.452, 0.939, "(b)", ha="left", va="bottom", fontsize=9)
89
90     ring_handle = Line2D([0], [0], color=blue, lw=1.65, marker="o", mfc="white",
91 ms=3.5)
92     fixed_handle = Line2D([0], [0], color=orange, lw=1.1, ls="--", marker="x", ms
93 =4.5)
94     fig.legend([ring_handle, fixed_handle], [r"材料环 $\xi=0.5$", r"固定位置 $r
95 =1.5\,\mathrm{cm}$"],
96             loc="lower center", bbox_to_anchor=(0.722, 0.017), ncol=2, frameon=
97 False,
98             fontsize=8, handlelength=1.6, handletextpad=0.4, columnspacing=1.0)
99     return fig
100 def render():
101     a=np.array(obs()["q4_radius_time_s_cm"])
102     selected=[float(a[a[:,0]==t,1].item()) for t in [0,21600]]
103     export(draw(a[:,0]/3600,a[:,1],selected),"fig06_q4_coordinates",dpi=360)

```

B.9 问题一径向剖面图

绘制五个时刻的温度与含水率径向分布。

程序 9 code/figure_core.py

```

1  """Portable functions extracted from the team's existing figure source.
2  No solver is imported. See figures/README.md for scope and data provenance.
3  """
4  from pathlib import Path
5  import json
6  import matplotlib
7  matplotlib.use("Agg")
8  import matplotlib.pyplot as plt
9  from matplotlib import font_manager
10 from matplotlib.colors import LinearSegmentedColormap, Normalize, to_rgb
11 from matplotlib.cm import ScalarMappable
12 from matplotlib.lines import Line2D
13 from matplotlib.patches import Circle
14 from matplotlib.text import Text

```

程序9 code/figure_core.py (续)

```

15 import matplotlib.patheffects as path_effects
16 import numpy as np
17 DATA = Path(__file__).resolve().parents[1] / "figure_data"
18 OUT = Path.cwd() / "redrawn_figures"
19
20 def obs():
21     return json.loads((DATA/"radius_observations.json").read_text(encoding="utf-8"
22     ))
23
24 def export(fig, name, dpi=300, transparent=False):
25     OUT.mkdir(parents=True, exist_ok=True)
26     for ext in ("pdf", "svg", "png"):
27         fig.savefig(OUT/f"{name}.{ext}", dpi=dpi, transparent=transparent)
28     plt.close(fig)
29
30 COLORS = ["#0072BD", "#D95319", "#3A923A", "#7E2F8E", "#333333"]
31 STYLES = ["-", "--", "-.", ":", (0, (5, 1, 1, 1))]
32 MARKERS = ["o", "s", "^", "D", "v"]
33 WIDTH = 15.8/2.54
34
35 def save(fig, name):
36     upright_ylabels(fig)
37     export(fig, name)
38
39 def axes_pair(height=2.8):
40     fig, axs = plt.subplots(1, 2, figsize=(WIDTH, height), layout="constrained")
41     for ax in axs:
42         ax.grid(True, color=".87", linewidth=.4)
43         ax.set_axisbelow(True)
44     return fig, axs
45
46 def tag(ax, letter, name):
47     # Descriptive titles belong in the adjacent manuscript text/caption.
48     ax.set_title(f"({letter})", loc="left", pad=7, fontweight="bold")
49
50 def upright_ylabels(fig):
51     """Stack Chinese names, keeping units and mathematical expressions intact."""
52     for ax in fig.axes:
53         label = ax.get_ylabel()
54         if " / " not in label:
55             continue
56         name, unit = label.split(" / ", 1)
57         if not all("\u4e00" <= char <= "\u9fff" for char in name):
58             continue
59         ax.set_ylabel("\n".join(name) + "\n" + unit, rotation=0,
60             ha="right", va="center", labelpad=10)
61         ax.yaxis.label.set_multialignment("center")
62         ax.yaxis.label.set_linespacing(1.05)
63
64 def line(ax, x, y, i, label, markevery=None, **kw):

```

程序9 code/figure_core.py (续)

```

64     ax.plot(x, y, color=COLORS[i % 5], linestyle=STYLES[i % 5],
65             marker=MARKERS[i % 5] if markevery else None,
66             markevery=markevery, markersize=3.2, markerfacecolor="white",
67             markeredgewidth=.9, label=label, **kw)
68
69 def fig03():
70     p = DATA/"q1_profiles.npz"
71     fig, axs = axes_pair(2.8)
72     with np.load(p) as z:
73         # Five saved times use distinct markers and line styles.
74         for time, style_index in [(100, 0), (600, 1), (900, 3), (1200, 4), (1800,
75                                     2)]:
76             k = np.flatnonzero(z["time_s"] == time).item()
77             for ax, field in zip(axs, ["temperature_C", "moisture_kg_kg"]):
78                 line(ax, z["radius_m"]*100, z[field][k], style_index, f"{time} s",
79                     markevery=2)
80             for ax, title, ylabel in zip(axs, ["温度径向剖面", "含水率径向剖面"], ["温度 / °C",
81                                     "干基含水率 / (kg/kg)"]):
82                 tag(ax, "a" if ax is axs[0] else "b", title)
83                 ax.set(xlim=(-.04, 2.04), xlabel="半径 / cm", ylabel=ylabel)
84                 ax.set_xticks([0, .5, 1, 1.5, 2])
85                 ax.legend(frameon=False, loc="upper left" if ax is axs[0] else "lower left",
86                             ncol=2, handlelength=2.7, columnspacing=1)
87     save(fig, 'fig03_q1_profiles')

```

B.10 问题二时空分布图

读取前三小时的温度与含水率场，绘制对应时空分布。

程序10 code/figure_fields.py

```

1  """Portable functions extracted from the team's existing figure source.
2  No solver is imported. See figures/README.md for scope and data provenance.
3  """
4  from pathlib import Path
5  import json
6  import matplotlib
7  matplotlib.use("Agg")
8  import matplotlib.pyplot as plt
9  from matplotlib import font_manager
10 from matplotlib.colors import LinearSegmentedColormap, Normalize, to_rgb
11 from matplotlib.cm import ScalarMappable
12 from matplotlib.lines import Line2D
13 from matplotlib.patches import Circle
14 from matplotlib.text import Text
15 import matplotlib.path_effects as path_effects
16 import numpy as np
17 DATA = Path(__file__).resolve().parents[1] / "figure_data"

```

程序 10 code/figure_fields.py (续)

```

18 OUT = Path.cwd() / "redrawn_figures"
19
20 def obs():
21     return json.loads((DATA/"radius_observations.json").read_text(encoding="utf-8"
22     ))
23
24 def export(fig, name, dpi=300, transparent=False):
25     OUT.mkdir(parents=True, exist_ok=True)
26     for ext in ("pdf", "svg", "png"):
27         fig.savefig(OUT/f"{name}.{ext}", dpi=dpi, transparent=transparent)
28     plt.close(fig)
29
30 WIDTH_CM=15.8
31 HEIGHT_IN=3.2
32 PALETTE_POSITIONS=[0,.25,.5,.75,1]
33 TEMPERATURE_COLORS=["#ffffaf1", "#fee0c5", "#f9b39d", "#e77968", "#b8443f"]
34 MOISTURE_COLORS=["#eaf3f7", "#d6eff5", "#99d3e1", "#469dbf", "#155c88"]
35 MOISTURE_LIMITS=[0,2.55]
36 CONTOUR_LEVELS=[[35,40,45,48],[1.5,2,2.4]]
37 LABEL_POSITIONS=[[(.65,1.1),(.95,1.65),(1.45,1.1),(1.85,1.65)],[(1.5,1.8)
38     ,(1.8,1.25),(1.1,.75)]]
39
40 def palette(name: str, colors: list[str]) -> LinearSegmentedColormap:
41     return LinearSegmentedColormap.from_list(
42         name, list(zip(PALETTE_POSITIONS, colors)), N=256)
43
44 def draw_fields(time_h: np.ndarray, radius_cm: np.ndarray,
45     fields: list[np.ndarray], textfree: bool = False) -> plt.Figure:
46     cmaps = [palette("q2_warm_coral", TEMPERATURE_COLORS),
47         palette("shared_moisture_lake", MOISTURE_COLORS)]
48     limits = [[float(fields[0].min()), float(fields[0].max())], MOISTURE_LIMITS]
49     fig, axes = plt.subplots(1, 2, figsize=(WIDTH_CM / 2.54, HEIGHT_IN),
50         layout="constrained")
51     dt = float(time_h[1] - time_h[0])
52     dr = float(radius_cm[1] - radius_cm[0])
53     # Saved node values are pixel centers, so the image edges lie half a cell out.
54     extent = [float(time_h[0] - dt / 2), float(time_h[-1] + dt / 2),
55         float(radius_cm[0] - dr / 2), float(radius_cm[-1] + dr / 2)]
56     for j, (ax, values, cmap, bounds) in enumerate(zip(axes, fields, cmaps, limits
57     )):
58         mesh = ax.imshow(values, origin="lower", aspect="auto", extent=extent,
59             interpolation="bilinear", interpolation_stage="data",
60             cmap=cmap, vmin=bounds[0], vmax=bounds[1],
61             rasterized=True)
62         ax.set(xlim=(0, 3), ylim=(0, 2))
63         contour_color = "#6e332c" if j == 0 else "#153e57"
64         contours = ax.contour(time_h, radius_cm, values, levels=CONTOUR_LEVELS[j],
65             colors=[contour_color], linewidths=0.7)
66         labels = ax.clabel(contours, inline=False, fontsize=7, fmt="%g",
67             manual=LABEL_POSITIONS[j])
68     for label in labels:

```

程序 10 code/figure_fields.py (续)

```

65         label.set_rotation(0)
66         label.set_path_effects([
67             path_effects.withStroke(linewidth=2.0, foreground="#ffffff", alpha
=0.9)])
68         cbar = fig.colorbar(mesh, ax=ax, orientation="horizontal", fraction=0.075,
69                             pad=0.13, aspect=24)
70         cbar.set_label(["温度 / °C", "干基含水率 / (kg/kg)"][j], fontsize=8)
71         cbar.ax.tick_params(labelsize=8)
72         if j == 1:
73             cbar.set_ticks([0, 0.5, 1, 1.5, 2, 2.55],
74                             labels=["0", "0.5", "1", "1.5", "2", "2.55"])
75         ax.set_xlabel("时间 / h")
76         ax.set_xticks([0, 1, 2, 3])
77         ax.set_yticks([0, 0.5, 1, 1.5, 2])
78         ax.set_ylabel("半\ncm", rotation=0, ha="right", va="center", labelpad
=10)
79         ax.yaxis.label.set_multialignment("center")
80         ax.yaxis.label.set_linespacing(1.05)
81         ax.set_title(f"{'ab'[j]}", loc="left", pad=7, fontweight="bold")
82         ax.grid(False)
83         # Use the same output-resolution layout for the full and text-free versions.
84         fig.set_dpi(300)
85         fig.canvas.draw()
86         fig.set_layout_engine(None)
87         if textfree:
88             # Keep every graphical element (including contours, ticks, axes and
89             # colorbars) in place; hide only the text after the layout is frozen.
90             for text in fig.findobj(Text):
91                 text.set_visible(False)
92         return fig
93     def render():
94         with np.load(DATA/"q2_fields.npz", allow_pickle=False) as z:
95             fig=draw_fields(z["time_s"]/3600,z["radius_m"]*100,[z[k].T for k in ["
temperature_C", "moisture_kg_kg"]])
96             export(fig,"fig04_q2_fields_sample")

```

B.11 空间离散、扩散系数分解与达标进程图

构造圆环控制体示意；从主解状态分解扩散系数变化，并提取各规定半径的达标进程。

程序 11 code/figure_restructured.py

```
1  """Draw three explanatory figures from compact, unrounded numerical data.
2
3  Portable rendering:
4      python figure_restructured.py --data-dir compact --out-dir figures
5      python figure_restructured.py --data-dir compact --out-dir figures --only 2-2
6  Explicit extraction from an original run (does not execute a solver):
7      python figure_restructured.py --source run.npz --data-dir compact --extract-only
8  Dependencies: Python 3.10+, NumPy and Matplotlib; supply --font-path if needed.
9  """
10 from __future__ import annotations
11
12 import argparse
13 import hashlib
14 import json
15 from pathlib import Path
16
17 import numpy as np
18 import matplotlib
19 matplotlib.use("Agg")
20 import matplotlib.pyplot as plt
21 from matplotlib import font_manager
22 from matplotlib.colors import to_rgba
23 from matplotlib.lines import Line2D
24 from matplotlib.patches import Circle, Rectangle, Patch
25 from matplotlib.text import Text, Annotation
26
27 DATA_NAME = "restructured_figure_data.json"
28 WIDTH_CM = 15.8
29 BLUE, PALE_BLUE = "#286F8C", "#E4EFF3"
30 WARM, INK, GRAY = "#C76B50", "#263D49", "#74838A"
31 NAMES = {"1-1": "fig01_annular_control_volumes",
32          "2-2": "fig09_q2_diffusivity_decomposition",
33          "3-1": "fig05_q3_completion_progress"}
34
35
36 def write_json(path: Path, value: dict) -> None:
37     path.parent.mkdir(parents=True, exist_ok=True)
38     path.write_text(json.dumps(value, ensure_ascii=False, indent=2) + "\n",
39                     encoding="utf-8")
40
41 def digest(path: Path) -> str:
42     return hashlib.sha256(path.read_bytes()).hexdigest()
43
```

程序 11 code/figure_restructured.py (续)

```

44
45 def extract_data(source_path: Path, data_dir: Path, provenance_path: Path | None =
    None) -> dict:
46     """Reduce existing arrays to all quantitative values required by the plots."""
47     source_path = source_path.resolve()
48     with np.load(source_path, allow_pickle=False) as z:
49         time = z["time_s"].copy()
50         radius = z["radius_m"].copy()
51         moisture = z["moisture_kg_kg"].copy()
52         maximum = z["max_moisture_kg_kg"].copy()
53         node_count = int(z["mesh_radius_m"].size)
54         assert time.size > 10800 and np.array_equal(time, np.arange(time.size)), "
            Expected a continuous integer-second trajectory from zero through at least 3 h.
            "
55         np.testing.assert_allclose(radius, np.arange(21) * .001, rtol=0, atol=2e
            -18)
56         assert moisture.shape == (time.size, 21)
57         assert np.isfinite(moisture).all() and np.isfinite(maximum).all()
58         assert (moisture > 0).all()
59         assert np.all(maximum >= moisture.max(axis=1) - 2e-15)
60         sample_seconds = np.array([1800, 5400, 10800])
61         c = moisture[sample_seconds][:, [0, -1]]
62         t = z["temperature_C"][sample_seconds][:, [0, -1]]
63
64         heat = 3850 * (1 / (28 + 273.15) - 1 / (t + 273.15))
65         drying = -.45 * (1 / c - 1 / 2.55)
66         diffusivity = 2.4e-3 * np.exp(-.45 / c - 3850 / (t + 273.15))
67         d0 = 2.4e-3 * np.exp(-.45 / 2.55 - 3850 / (28 + 273.15))
68         net = np.log(diffusivity / d0)
69         np.testing.assert_allclose(heat + drying, net, rtol=0, atol=5e-15)
70         assert (heat > 0).all() and (drying < 0).all()
71
72         threshold = .15
73         first = np.array([np.flatnonzero(moisture[:, j] < threshold)[0] for j in range
            (21)])
74         end_index = int(np.flatnonzero(maximum < threshold)[0])
75         end = int(time[end_index])
76         assert int(first.max()) <= end, "All-node completion cannot precede sampled-
            position completion."
77         assert np.all(maximum[end_index:] < threshold)
78         rows = []
79         for j, index in enumerate(first):
80             assert index > 0 and moisture[index-1, j] >= threshold
81             assert np.all(moisture[index:, j] < threshold), "Threshold recrossing
            requires segmented bands."
82             rows.append({"radius_cm": float(100 * radius[j]),
83                         "first_strict_second": int(time[index]),
84                         "previous_moisture_kg_kg": float(moisture[index-1, j]),
85                         "first_moisture_kg_kg": float(moisture[index, j])})
86         data = {

```

程序 11 code/figure_restructured.py (续)

```

87     "schema_version": "1.0", "source_file": source_path.name,
88     "diffusivity": {
89         "time_s": sample_seconds.tolist(), "positions": ["中心", "表面"],
90         "temperature_C": t.tolist(), "moisture_kg_kg": c.tolist(),
91         "initial_temperature_C": 28., "initial_moisture_kg_kg": 2.55,
92         "heat_log_contribution": heat.tolist(), "drying_log_contribution":
drying.tolist(),
93         "net_log_ratio": net.tolist(), "diffusivity_m2_s": diffusivity.tolist
(),
94         "initial_diffusivity_m2_s": float(d0),
95         "interpretation": "Exact algebraic decomposition along one main
trajectory; not independent causal effects or alternate simulations."
96     },
97     "completion": {
98         "threshold_kg_kg": threshold, "sampling_interval_s": 1,
99         "checked_time_range_s": [int(time[0]), int(time[-1])],
100        "sampled_positions": rows, "threshold_recrossings": 0,
101        "all_node_count": node_count, "all_node_first_strict_second": end,
102        "all_node_previous_maximum_kg_kg": float(maximum[end_index-1]),
103        "all_node_first_maximum_kg_kg": float(maximum[end_index]),
104        "interpretation": f"Bars classify integer-second samples at 21
requested output radii; the final stopping time separately uses the maximum
over all {node_count} solver nodes."
105    }
106 }
107 write_json(data_dir / DATA_NAME, data)
108 provenance = {
109     "source": {"path": str(source_path), "sha256": digest(source_path)},
110     "compact_file": DATA_NAME, "compact_sha256": digest(data_dir / DATA_NAME),
111     "extraction_script_sha256": digest(Path(__file__)), "solver_executed":
False,
112     "identity_residual": float(np.max(np.abs(heat + drying - net))),
113     "threshold_checks": {"positions": 21, "checked_until_s": int(time[-1]),
114                          "sample_interval_s": 1, "recrossings": 0,
115                          "all_node_final_s": end},
116     "rounding": "All numerical inputs retained as float64; only plot labels
are formatted."
117 }
118 if provenance_path is not None:
119     write_json(provenance_path, provenance)
120 return data
121
122
123 def configure(font_path: str | None) -> str:
124     if font_path:
125         font_manager.fontManager.addfont(font_path)
126         name = font_manager.FontProperties(fname=font_path).get_name()
127     else:
128         fonts = {f.name for f in font_manager.fontManager.ttflist}

```

程序 11 code/figure_restructured.py (续)

```

129     name = next((x for x in ["Microsoft YaHei", "Noto Sans CJK SC", "Source
Han Sans SC", "SimHei"] if x in fonts), None)
130     if name is None:
131         raise RuntimeError("Chinese font unavailable: provide --font-path with
a Chinese font file.")
132     plt.rcParams.update({
133         "font.family": name, "font.size": 9, "axes.labelsize": 9,
134         "xtick.labelsize": 8, "ytick.labelsize": 8, "legend.fontsize": 8,
135         "axes.linewidth": .8, "lines.linewidth": 1.8,
136         "mathtext.fontset": "stix", "axes.unicode_minus": False,
137         "xtick.direction": "out", "ytick.direction": "out",
138         "pdf.fonttype": 42, "ps.fonttype": 42, "svg.fonttype": "none",
139         "figure.facecolor": "white", "savefig.facecolor": "white",
140         "path.simplify": False,
141     })
142     return name
143
144
145 def gray(color):
146     r, g, b, a = to_rgba(color)
147     y = .2126*r + .7152*g + .0722*b
148     return (y, y, y, a)
149
150
151 def save_bundle(fig, key: str, out_dir: Path, qa: bool, font: str, metadata: dict)
-> dict:
152     name = NAMES[key]
153     out_dir.mkdir(parents=True, exist_ok=True)
154     fig.canvas.draw()
155     outputs = {}
156     for ext in ["png", "pdf", "svg"]:
157         path = out_dir / f"{name}.{ext}"
158         fig.savefig(path, dpi=300)
159         outputs[ext] = {"file": path.name, "sha256": digest(path)}
160     if qa:
161         qa_dir = out_dir / "qa"
162         qa_dir.mkdir(exist_ok=True)
163         texts = list(fig.findobj(Text))
164         visible = [x.get_visible() for x in texts]
165         strings = [x.get_text() for x in texts]
166         for x in texts:
167             if isinstance(x, Annotation):
168                 x.set_text("") # retain geometric arrows attached to annotation
169             artists
170             else:
171                 x.set_visible(False)
172         fig.savefig(qa_dir / f"{name}_no_text.png", dpi=300)
173         for x, vis, string in zip(texts, visible, strings):
174             x.set_visible(vis)
175             x.set_text(string)

```

程序 11 code/figure restructured.py (续)

```

175     for item in fig.findobj(Line2D):
176         for getter, setter in [(item.get_color, item.set_color),
177                                (item.get_markeredgcolor, item.
set_markeredgcolor),
178                                (item.get_markerfacecolor, item.
set_markerfacecolor)]:
179             try:
180                 setter(gray(getter()))
181             except (ValueError, TypeError):
182                 pass
183     for item in fig.findobj(Patch):
184         item.set_facecolor(gray(item.get_facecolor()))
185         item.set_edgecolor(gray(item.get_edgecolor()))
186     for item in texts:
187         item.set_color(gray(item.get_color()))
188         if isinstance(item, Annotation) and item.arrow_patch is not None:
189             item.arrow_patch.set_facecolor(gray(item.arrow_patch.get_facecolor
190             ()))
191             item.arrow_patch.set_edgecolor(gray(item.arrow_patch.get_edgecolor
192             ()))
193     for collection in fig.findobj(matplotlib.collections.Collection):
194         for getter, setter in [(collection.get_facecolors, collection.
set_facecolors),
195                                (collection.get_edgecolors, collection.
set_edgecolors)]:
196             values = getter()
197             if values.size:
198                 setter([gray(x) for x in values])
199     fig.savefig(qa_dir / f"{name}_gray.png", dpi=300)
200     metadata.update({"id": key, "name": name, "outputs": outputs,
201                    "width_cm": WIDTH_CM, "height_cm": float(fig.get_figheight()
202                    *2.54),
203                    "font": font, "font_size_pt": 9, "solver_executed": False})
204     plt.close(fig)
205     return metadata
206
207 def annular_figure():
208     fig = plt.figure(figsize=(WIDTH_CM/2.54, 7.9/2.54))
209     a = fig.add_axes([.035, .12, .34, .80])
210     b = fig.add_axes([.445, .15, .53, .76])
211     for ax in [a, b]:
212         ax.axis("off")
213     a.set_aspect("equal")
214     a.set_xlim(-1.13, 1.25); a.set_ylim(-1.2, 1.25)
215     # The five displayed node locations only illustrate node-centred geometry.
216     faces = np.array([0., .125, .375, .625, .875, 1.])
217     colors = [PALE_BLUE, "#F3F5F5", "#B9D7E1", "#F3F5F5", "#F4E3DB"]
218     for j in reversed(range(5)):
219         a.add_patch(Circle((0, 0), faces[j+1], fc=colors[j], ec="none"))

```

程序 11 code/figure_restructured.py (续)

```

218     for face in faces[1:-1]:
219         a.add_patch(Circle((0, 0), face, fill=False, ec=GRAY, lw=.7, ls=(0, (3, 2)
)))
220     a.add_patch(Circle((0, 0), 1, fill=False, ec=INK, lw=1.5))
221     a.plot([0, 1], [0, 0], color=INK, lw=1.2)
222     a.plot(np.linspace(0, 1, 5), np.zeros(5), "o", ms=4.4, color=INK, mec="white",
mew=.7)
223     a.text(-.065, -.095, "0", ha="right", va="top")
224     a.annotate("", (1.11, 0), (1.0, 0), arrowprops={"arrowstyle": "->", "lw": 1.1,
"color": INK})
225     a.text(1.15, 0, "$r$", va="center")
226     fig.text(.04, .945, "(a)", fontsize=10, fontweight="bold")
227
228     b.set_xlim(-.2, 4.75); b.set_ylim(-1.04, 1.4)
229     fig.text(.445, .945, "(b)", fontsize=10, fontweight="bold")
230     xfaces = faces * 4
231     for j in range(5):
232         b.add_patch(Rectangle((xfaces[j], -.28), xfaces[j+1]-xfaces[j], .56,
fc=colors[j], ec="none"))
233
234     b.plot([0, 4], [-.28, -.28], color=INK, lw=.9)
235     b.plot([0, 4], [.28, .28], color=INK, lw=.9)
236     for face in xfaces[1:-1]:
237         b.plot([face, face], [-.42, .43], color=GRAY, ls=(0, (3, 2)), lw=.9)
238         b.plot([0, 0], [-.28, .28], color=INK, lw=1.3)
239         b.plot([4, 4], [-.42, .43], color=INK, lw=1.7)
240         b.plot([0, 4], [0, 0], color=INK, lw=1.2)
241         b.plot(np.arange(5), np.zeros(5), "o", ms=5, color=INK, mec="white", mew=.8)
242         for x, label in zip(range(5), ["$r_0$", "$r_1$", "$r_i$", "$r_{i+1}$", "$r_N
=R$"]):
243             b.text(x, -.58, label, ha="center", va="top", fontsize=8.5)
244             b.text(1.5, .55, "$r_{i-1/2}$", ha="center", fontsize=8.5)
245             b.text(2.5, .55, "$r_{i+1/2}$", ha="center", fontsize=8.5)
246             b.annotate("", (1.5, .44), (1.5, .52), arrowprops={"arrowstyle": "-", "color":
GRAY})
247             b.annotate("", (2.5, .44), (2.5, .52), arrowprops={"arrowstyle": "-", "color":
GRAY})
248             for x0, x1, y, text in [(1., 2., -1.00, r"$\Delta r$"), (3.5, 4., .91, r"$\
Delta r/2$")]:
249                 b.plot([x0, x1], [y, y], color=INK, lw=1.)
250                 b.plot([x0, x0], [y-.045, y+.045], color=INK, lw=1.)
251                 b.plot([x1, x1], [y-.045, y+.045], color=INK, lw=1.)
252                 b.text((x0+x1)/2, y+.08, text, ha="center", fontsize=8.5)
253             b.annotate("", (4.7, .02), (4.05, .02), arrowprops={"arrowstyle": "-|>", "lw":
2.1, "color": BLUE, "mutation_scale": 13})
254             b.text(3.85, .52, "$h_m(C_N-C_{eq})$", ha="center", fontsize=8)
255             b.text(.1, 1.12, r"$C_0=C_1=\cdots=C_N=C_{init}$", ha="left", fontsize=9)
256             fig.legend(handles=[Line2D([], [], ls="none", marker="o", color=INK, ms=4.5,
label="节点"),
257                             Line2D([], [], color=GRAY, ls="--", lw=1., label="控制体界
面")],

```

程序 11 code/figure_restructured.py (续)

```

258         Patch(facecolor="#F4E3DB", edgecolor="none", label="表面半
控制体"]],
259         loc="lower center", bbox_to_anchor=(.52, -.01), ncol=3, frameon=
False)
260     return fig, {"data_driven_visual": False,
261                 "scope": "Conceptual Q1 radial node-centred control-volume
geometry, not a simulation result.",
262                 "mapping": "Solid dots are radial nodes; dashed circles/lines are
midpoint faces. Surface node is exactly at R, on the outer edge of its half-
width volume. Uniform initial moisture makes all internal face fluxes zero; the
arrow is the initial outward moisture exchange. Circle area is geometric, with
displayed node count schematic."}
263
264
265 def diffusivity_figure(data: dict):
266     d = data["diffusivity"]
267     heat, drying, net = (np.asarray(d[key]) for key in ["heat_log_contribution", "
drying_log_contribution", "net_log_ratio"])
268     c, t = np.asarray(d["moisture_kg_kg"]), np.asarray(d["temperature_C"])
269     reference = np.log(2.4e-3 * np.exp(-.45/c-3850/(t+273.15))) / d["
initial_diffusivity_m2_s"])
270     np.testing.assert_allclose(heat + drying, reference, rtol=0, atol=5e-15)
271     np.testing.assert_allclose(net, reference, rtol=0, atol=5e-15)
272     fig, axes = plt.subplots(1, 2, figsize=(WIDTH_CM/2.54, 7.3/2.54), sharex=True,
sharey=True)
273     fig.subplots_adjust(left=.095, right=.98, top=.84, bottom=.25, wspace=.18)
274     y = np.array([2., 1., 0.])
275     for j, ax in enumerate(axes):
276         ax.barh(y, heat[:, j], height=.48, color=WARM, edgecolor="white", lw=.6)
277         ax.barh(y, drying[:, j], height=.48, color=BLUE, edgecolor=INK, lw=.4,
hatch="///")
278         ax.plot(net[:, j], y, ls="none", marker="D", color=INK, mec="white", mew
=.85, ms=5.6, zorder=4)
279         ax.axvline(0, color=INK, lw=1.)
280         ax.set_axisbelow(True)
281         ax.grid(axis="x", color="#DAE0E3", lw=.55)
282         for side in ["top", "right", "left"]:
283             ax.spines[side].set_visible(False)
284         ax.tick_params(axis="y", length=0)
285         ax.set_yticks(y, [f"{x/3600:g} h" for x in d["time_s"]])
286         ax.set_ylim(-.55, 2.55)
287         ax.set_xlim(-.4, 1.02)
288         ax.set_xticks([-0.3, 0, .3, .6, .9])
289         ax.text(0, 1.10, f"({'ab'[j]}) {d['positions'][j]}", transform=ax.
transAxes, fontweight="bold")
290         ax.set_xlabel(r"$\ln(D/D_0)$ 的分解")
291         axes[1].tick_params(axis="y", labelleft=True)
292         fig.legend(handles=[Patch(facecolor=WARM, label="升温项"),
293                             Patch(facecolor=BLUE, edgecolor=INK, hatch="///", label="
失水项")],

```

程序 11 code/figure_restructured.py (续)

```

294         Line2D([], [], marker="D", ls="none", color=INK, mec="
white", ms=5, label="净变化")),
295         ncol=3, frameon=False, loc="lower center", bbox_to_anchor=(.54,
.005))
296     return fig, {"data_driven_visual": True, "source_file": data["source_file"],
297                 "quantity": "Natural logarithm of local diffusivity relative to
its uniform initial value; dimensionless.",
298                 "sampling": "0.5, 1.5 and 3 h, centre and surface.",
299                 "interpretation": d["interpretation"],
300                 "identity_residual": float(np.max(np.abs(heat+drying-reference)))
}
301
302
303 def completion_figure(data: dict):
304     d = data["completion"]
305     rows = d["sampled_positions"]
306     r = np.array([x["radius_cm"] for x in rows])
307     first = np.array([x["first_strict_second"] for x in rows])
308     end = d["all_node_first_strict_second"] / 3600
309     assert all(x["previous_moisture_kg_kg"] >= .15 > x["first_moisture_kg_kg"] for
x in rows)
310     assert d["all_node_previous_maximum_kg_kg"] >= .15 > d["
all_node_first_maximum_kg_kg"]
311     fig, ax = plt.subplots(figsize=(WIDTH_CM/2.54, 9.5/2.54))
312     fig.subplots_adjust(left=.12, right=.96, bottom=.19, top=.91)
313     ax.barh(r, end, height=.078, color=PALE_BLUE, edgecolor="none")
314     ax.barh(r, first/3600, height=.078, color=BLUE, edgecolor="none")
315     ax.plot(first/3600, r, ls="none", marker="o", ms=3.1, color=INK, mec="white",
mew=.5)
316     ax.axvline(end, color=WARM, lw=1.7, ls=(0, (4, 2)))
317     ax.set_axisbelow(True)
318     ax.grid(axis="x", color="#DAE0E3", lw=.55)
319     ax.set_ylim(-.09, 2.12)
320     ax.set_xlim(0, end + 1.4)
321     ax.set_yticks([0, .5, 1, 1.5, 2.], ["0", "0.5", "1.0", "1.5", "2.0"])
322     step = 12. if end >= 48 else (6. if end >= 24 else (3. if end >= 12 else 1.))
323     interior_ticks = np.arange(0, end, step)
324     interior_ticks = interior_ticks[(end-interior_ticks >= .35*step) | (
interior_ticks == 0)]
325     ax.set_xticks([*interior_ticks, end], [*f"{x:g}" for x in interior_ticks], f"
{end:.2f}")
326     ax.set_xlabel("时间 / h")
327     ax.set_ylabel("半径\ncm", rotation=0, labelpad=18, va="center")
328     for side in ["top", "right"]:
329         ax.spines[side].set_visible(False)
330     ax.text(end-.7, 2.095, "全域达标", ha="right", va="bottom", fontsize=8, color=
WARM)
331     fig.legend(handles=[Patch(facecolor=BLUE, label=r"$C\geq 0.15$"),
332                         Patch(facecolor=PALE_BLUE, edgecolor=GRAY, linewidth=.4,
label="$C<0.15$"),

```

程序 11 code/figure_restructured.py (续)

```

333         Line2D([], [], ls="none", marker="o", ms=4, color=INK,
label="首次严格达标秒"]],
334         ncol=3, frameon=False, loc="lower center", bbox_to_anchor=(.54,
.018))
335     return fig, {"data_driven_visual": True, "source_file": data["source_file"],
336                 "interpretation": d["interpretation"],
337                 "threshold_kg_kg": d["threshold_kg_kg"],
338                 "all_node_first_strict_second": d["all_node_first_strict_second"]
},
339                 "no_recrossing_check": {"through_s": d["checked_time_range_s"]
][1], "recrossings": d["threshold_recrossings"]},
340                 "time_display": "Hours on axis, underlying bar boundaries are
integer seconds; the rounded final tick label is not a new stopping time."}
341
342
343 def main():
344     parser = argparse.ArgumentParser(description=__doc__, formatter_class=argparse
.RawDescriptionHelpFormatter)
345     parser.add_argument("--data-dir", type=Path, required=True)
346     parser.add_argument("--out-dir", type=Path)
347     parser.add_argument("--source", type=Path, help="Explicitly extract compact
data from an existing original NPZ.")
348     parser.add_argument("--provenance", type=Path, help="Optional separate
extraction provenance JSON.")
349     parser.add_argument("--extract-only", action="store_true")
350     parser.add_argument("--only", nargs="+", choices=list(NAMES), default=list(
NAMES))
351     parser.add_argument("--font-path")
352     parser.add_argument("--qa", action="store_true", help="Also create text-hidden
and grayscale audit PNGs.")
353     args = parser.parse_args()
354     if args.source:
355         extract_data(args.source, args.data_dir, args.provenance)
356     if args.extract_only:
357         if not args.source:
358             parser.error("--extract-only requires --source")
359         print(f"Prepared {DATA_NAME}; no solver was executed.")
360         return
361     if args.out_dir is None:
362         parser.error("--out-dir is required for rendering")
363     data_path = args.data_dir / DATA_NAME
364     data = json.loads(data_path.read_text(encoding="utf-8"))
365     font = configure(args.font_path)
366     generators = {"1-1": lambda: annular_figure(),
367                  "2-2": lambda: diffusivity_figure(data),
368                  "3-1": lambda: completion_figure(data)}
369     figures = []
370     for key in args.only:
371         fig, meta = generators[key]()
372         figures.append(save_bundle(fig, key, args.out_dir, args.qa, font, meta))

```

程序 11 code/figure_restructured.py (续)

```

373     write_json(args.out_dir / "figure_manifest.json", {
374         "schema_version": "1.0", "generator_file": Path(__file__).name,
375         "generator_sha256": digest(Path(__file__)), "compact_file": DATA_NAME,
376         "compact_sha256": digest(data_path), "figures": figures,
377         "qa_rendered": args.qa})
378     print(f"Rendered {len(figures)} figures with PDF, SVG and PNG outputs.")
379
380
381 if __name__ == "__main__":
382     main()

```

B.12 问题四含水率截面图

将六个时刻的完整径向含水率快照映射到同一厘米尺度下的圆截面。

程序 12 code/figure_sections.py

```

1  """Portable functions extracted from the team's existing figure source.
2  No solver is imported. See figures/README.md for scope and data provenance.
3  """
4  from pathlib import Path
5  import json
6  import matplotlib
7  matplotlib.use("Agg")
8  import matplotlib.pyplot as plt
9  from matplotlib import font_manager
10 from matplotlib.colors import LinearSegmentedColormap, Normalize, to_rgb
11 from matplotlib.cm import ScalarMappable
12 from matplotlib.lines import Line2D
13 from matplotlib.patches import Circle
14 from matplotlib.text import Text
15 import matplotlib.path_effects as path_effects
16 import numpy as np
17 DATA = Path(__file__).resolve().parents[1] / "figure_data"
18 OUT = Path.cwd() / "redrawn_figures"
19
20 def obs():
21     return json.loads((DATA/"radius_observations.json").read_text(encoding="utf-8"
22 ))
23
24 def export(fig, name, dpi=300, transparent=False):
25     OUT.mkdir(parents=True, exist_ok=True)
26     for ext in ("pdf", "svg", "png"):
27         fig.savefig(OUT/f"{name}.{ext}", dpi=dpi, transparent=transparent)
28     plt.close(fig)
29
30 PALETTE=("#eaf3f7", "#d6eff5", "#99d3e1", "#469dbf", "#155c88")
31 THRESHOLD=.15
32 THRESHOLD_COLOR="#66395F"

```

程序 12 code/figure_sections.py (续)

```

32 FIELD_LIMITS=(0,2.55)
33 AXIS_LIMIT_CM=2.2
34 DISPLAY_RINGS=512
35 WIDTH_CM,HEIGHT_CM=15.8,11.5
36 def grey(color):
37     """Equal-RGB tone with the same Rec.709 luminance as the input colour."""
38     rgb = np.asarray(to_rgb(color))
39     linear = np.where(rgb <= 0.04045, rgb / 12.92, ((rgb + 0.055) / 1.055) ** 2.4)
40     lum = float(linear @ np.array([0.2126, 0.7152, 0.0722]))
41     tone = 12.92 * lum if lum <= 0.0031308 else 1.055 * lum ** (1 / 2.4) - 0.055
42     return (tone, tone, tone)
43
44 def make_figure(xi, frames, *, grayscale=False):
45     colors = [grey(c) for c in PALETTE] if grayscale else list(PALETTE)
46     cmap = LinearSegmentedColormap.from_list("q4_fixed_moisture", colors, N=1024)
47     norm = Normalize(*FIELD_LIMITS, clip=False)
48     threshold_color = grey(THRESHOLD_COLOR) if grayscale else THRESHOLD_COLOR
49     rim_color = grey("#47525A") if grayscale else "#47525A"
50     fig = plt.figure(figsize=(WIDTH_CM / 2.54, HEIGHT_CM / 2.54))
51     panel_width = 0.27
52     panel_height = panel_width * WIDTH_CM / HEIGHT_CM
53     # Identical square axes enforce a common centimetre scale; no autoscaling.
54     for i, frame in enumerate(frames):
55         row, col = divmod(i, 3)
56         left = (0.055, 0.365, 0.675)[col]
57         bottom = (0.570, 0.145)[row]
58         ax = fig.add_axes((left, bottom, panel_width, panel_height), facecolor="
none")
59         ax.set(xlim=(-AXIS_LIMIT_CM, AXIS_LIMIT_CM), ylim=(-AXIS_LIMIT_CM,
AXIS_LIMIT_CM))
60         ax.set_aspect("equal", adjustable="box")
61         ax.set_axis_off()
62         radius = frame["radius_cm"]
63         field = frame["moisture"]
64         # Draw nested filled disks from outside inward. The visible thin bands
65         # sample the retained 10241-node radial field at their midpoints.
66         # All disks and contours remain vector primitives in PDF and SVG.
67         bands = 1 if np.ptp(field) == 0 else DISPLAY_RINGS
68         edges = np.linspace(0, radius, bands + 1)
69         middle = (edges[:-1] + edges[1:]) / 2
70         values = np.interp(middle / radius, xi, field)
71         for edge, value in zip(edges[:0:-1], values[::-1]):
72             ax.add_patch(Circle((0, 0), float(edge), facecolor=cmap(norm(value)),
edgecolor="none", linewidth=0, antialiased=False))
73         ax.add_patch(Circle((0, 0), radius, fill=False, edgecolor=rim_color,
linewidth=0.95))
74         if frame["threshold_radius_cm"] is not None:
75             ax.add_patch(Circle((0, 0), frame["threshold_radius_cm"], fill=False,
edgecolor=threshold_color, linewidth=1.15,
linestyle=(0, (3.2, 1.6))))

```

程序 12 code/figure_sections.py (续)

```

79     time_label = f"{frame['time_s'] / 3600:g} h" if i != 5 else f"{frame['
time_s']/3600:.4f} h"
80     header = f"({chr(97 + i)}) {time_label} " + rf"$R={radius:.3f}\,\mathrm{{
cm}}$"
81     ax.text(0.5, 1.030, header, transform=ax.transAxes,
82             ha="center", va="bottom", fontsize=9)
83
84     # One contour key is sufficient; the solid rim is the actual material edge.
85     key = Line2D([0], [0], color=threshold_color, linewidth=1.15, linestyle=(0,
(3.2, 1.6)))
86     fig.legend([key], [r"$C=0.15$"], loc="center", bbox_to_anchor=(0.5, 0.152),
87                 frameon=False, fontsize=9, handlelength=2.7, handletextpad=0.7)
88     bar_ax = fig.add_axes((0.20, 0.098, 0.60, 0.024), facecolor="none")
89     cb = fig.colorbar(ScalarMappable(norm=norm, cmap=cmap), cax=bar_ax,
orientation="horizontal",
90                       ticks=[0, 0.5, 1.0, 1.5, 2.0, 2.55])
91     cb.ax.tick_params(labelsize=8, length=2.2, width=0.65, pad=2)
92     cb.ax.set_xticklabels(["0", "0.5", "1.0", "1.5", "2.0", "2.55"])
93     cb.outline.set_linewidth(0.65)
94     cb.set_label("干基含水率 / (kg/kg)", fontsize=9, labelpad=3)
95     return fig
96 def render():
97     frames=[]
98     with np.load(DATA/"q4_sections.npz",allow_pickle=False) as z:
99         xi=z["mesh_xi"]
100         for t,R,C in zip(z["snapshot_time_s"],z["snapshot_radius_m"],z["
moisture_snapshots"]):
101             assert np.isfinite(C).all() and np.max(np.diff(C))<1e-12
102             radius=100*float(R)
103             level=float(np.interp(.15,C[:,:-1],(radius*xi)[::-1])) if C.min()<.15<C
.max() else None
104             frames.append({"time_s":int(t),"radius_cm":radius,"moisture":C,"
threshold_radius_cm":level})
105         fig=make_figure(xi,frames)
106         export(fig,"fig07_q4_sections_sample",dpi=360,transparent=True)

```

B.13 数据图绘制入口

统一选择绘图对象、输入数据、字体与输出目录。

程序 13 code/redraw_figures.py

```
1 """Redraw seven current figures; the spacetime grid has an editable SVG source.
2 Read compact numerical arrays and write to a separate output directory.
3 """
4 from pathlib import Path
5 import argparse
6 import importlib
7 import subprocess
8 import sys
9 import matplotlib
10 matplotlib.use('Agg')
11 from matplotlib import font_manager
12 import matplotlib.pyplot as plt
13
14 def font_name(path=None):
15     if path:
16         font_manager.fontManager.addfont(str(path))
17         return font_manager.FontProperties(fname=str(path)).get_name()
18     candidates=['Microsoft YaHei', 'Noto Sans CJK SC', 'Source Han Sans SC', 'SimHei'
19 , 'PingFang SC']
20     installed={f.name for f in font_manager.fontManager.ttflist}
21     for name in candidates:
22         if name in installed:
23             return name
24     raise RuntimeError('请安装中文字体, 或以 --font 指定字体文件。')
25
26 def configure(font, kind):
27     plt.rcParams.update({'font.family':font, 'font.size':9, 'axes.labelsize':9,
28 'xtick.labelsize':8, 'ytick.labelsize':8, 'legend.fontsize':8, 'axes.
29 titlesize':9,
30 'mathtext.fontset':'stix', 'axes.unicode_minus':False, 'axes.linewidth':.85,
31 'lines.linewidth':1.8, 'xtick.direction':'in', 'ytick.direction':'in',
32 'pdf.fonttype':42, 'ps.fonttype':42, 'svg.fonttype':'none',
33 'figure.facecolor':'white', 'savefig.facecolor':'white'})
34     if kind=='coordinates':
35         plt.rcParams.update({'font.size':8.5, 'axes.linewidth':.8})
36     elif kind=='sections':
37         plt.rcParams.update({'figure.facecolor':'none', 'savefig.facecolor':'none',
38 'xtick.direction':'out', 'ytick.direction':'out'})
39
40 def main():
41     p=argparse.ArgumentParser(description=__doc__)
42     p.add_argument('--output', type=Path, required=True)
43     p.add_argument('--data', type=Path, default=Path(__file__).resolve().parents[1]/
44 'figure_data')
45     p.add_argument('--font', type=Path)
```

程序 13 code/redraw_figures.py (续)

```

44     p.add_argument('--only',nargs='*',choices=['1-1','1-3','2-1','2-2','3-1','4-1'
45     , '4-2'])
46     a=p.parse_args(); output=a.output.resolve()
47     baseline=Path(__file__).resolve().parents[1]/'figures'
48     if output==baseline.resolve() or baseline.resolve() in output.parents:
49         raise ValueError('重绘请使用 figures 以外的新目录, 保留随包图件。')
50     font=font_name(a.font)
51     selected=[x for x in ['1-1','2-2','3-1'] if not a.only or x in a.only]
52     if selected:
53         command=[sys.executable, '-B', str(Path(__file__).with_name('
figure_restructured.py')),
54                 '--data-dir', str(a.data.resolve()), '--out-dir', str(output), '--
only', *selected]
55         if a.font: command.extend(['--font-path', str(a.font.resolve())])
56         subprocess.run(command, check=True, timeout=180)
57         for module, label, function in [('core', '1-3', 'fig03'), ('fields', '2-1', 'render')
,
58                                     ('coordinates', '4-1', 'render'), ('sections', '4-2'
, 'render')]:
59             if a.only and label not in a.only: continue
60             configure(font, module)
61             m=importlib.import_module('figure_'+module); m.DATA=a.data.resolve(); m.
OUT=output
62             getattr(m, function)()
63             print('Rendered figure '+label)
64 if __name__=='__main__': main()

```